

In [438...

```

(* ===== *)
(* S113B *)
(* QUERY-EFFICIENT ACTIVE S4 STATE DISCOVERY *)
(* *)
(* DEVELOPMENT GOAL *)
(* ----- *)
(* Recover the same hidden S4 machine as S113A, but WITHOUT *)
(* querying the full 16-bit behavior signature for every *)
(* discovered successor. *)
(* *)
(* LEARNER KNOWS ONLY TASK-LEVEL STRUCTURE: *)
(* - alphabet = {0,1,2} *)
(* - query symbols = {0,1,2,3} *)
(* - each observable state acts as a bijection on 4 symbols *)
(* - oracle answers one binary final-output query at a time *)
(* *)
(* LEARNER IS NOT GIVEN: *)
(* - hidden S4 state *)
(* - hidden permutation *)
(* - hidden transition table *)
(* - hidden reachable-state count *)
(* - intermediate trace *)
(* *)
(* Query policy: *)
(* - maintain candidate behavioral bijections *)
(* - choose query minimizing worst remaining candidate set *)
(* - stop querying a prefix once exactly one behavior remains *)
(* *)
(* State search still stops ONLY at transition closure. *)
(* *)
(* TRUE S4 is used ONLY after freeze for audit. *)
(* ===== *)

ClearAll["Global`S113B*"];
ClearAll["Global`s113B*"];

s113BResult =
Catch[

(

(* ===== *)
(* 0. FAIL SAFE *)
(* ===== *)

S113BFail[msg_] := (
  Print["STOP: ", msg];
  Throw[$Failed, "S113B_STOP"];
);

Print["====="];

```

```

Print["S113B QUERY-EFFICIENT ACTIVE S4 STATE DISCOVERY"];
Print["====="];

(* ===== *)
(* 1. TASK INTERFACE *)
(* ===== *)

alphabetS113B =
  {0, 1, 2};

querySymbolsS113B =
  {0, 1, 2, 3};

allQueryPairsS113B =
  Tuples[
    querySymbolsS113B,
    2
  ];

queryIndexS113B =
  AssociationThread[
    querySymbolsS113B,
    Range[4]
  ];

Print[
  "Primitive alphabet = ",
  alphabetS113B
];

Print[
  "Query symbols = ",
  querySymbolsS113B
];

Print[
  "Possible binary queries per complete signature = ",
  Length[allQueryPairsS113B]
];

Print["DISCOVERY WILL NOT QUERY ALL 16 BITS BY DEFAULT."];
Print["LEARNER IS NOT GIVEN THE HIDDEN REACHABLE-STATE COUNT."];
Print["LEARNER RECEIVES FINAL BINARY Y ONLY."];
Print["TRUE S4 IS POST-HOC ONLY."];

(* ===== *)
(* 2. PROSPECTIVE LONG-SEQUENCE HOLDOUT INPUTS *)

```

```

(*)                                                                    *)
(* Inputs are generated before discovery.                             *)
(* No oracle labels are accessed until after freeze.                   *)
(* ===== *)

holdoutSeedS113B =
  1130402;

holdoutDepthsS113B =
  {
    12,
    20,
    40,
    80
  };

holdoutSequencesPerDepthS113B =
  100;

prospectiveHoldoutSeqsS113B =
  BlockRandom[

    SeedRandom[
      holdoutSeedS113B
    ];

    Flatten[
      Table[

        Table[

          <|

            "Depth" ->
              depth,

            "Sequence" ->
              RandomChoice[
                alphabetS113B,
                depth
              ]

          |>,

          {
            holdoutSequencesPerDepthS113B
          }
        ],

        {
          depth,
          holdoutDepthsS113B
        }
      ]
    ]
  ]

```

```

        }
    ],
    1
]
];

Print[
    "Holdout sequences created = ",
    Length[
        prospectiveHoldoutSeqsS113B
    ]
];

Print[
    "Holdout depths = ",
    holdoutDepthsS113B
];

Print["HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED."];

(* ===== *)
(* 3. HIDDEN ENVIRONMENT *)
(* *)
(* Conceptually environment-only. *)
(* Learner may call ONLY S113BOracleY. *)
(* ===== *)

S113BEnvOp[0] :=
    {2, 1, 3, 4};

S113BEnvOp[1] :=
    {1, 3, 2, 4};

S113BEnvOp[2] :=
    {1, 2, 4, 3};

S113BEnvStep[
    hiddenState_List,
    token_Integer
] :=
    S113BEnvOp[token][[
        hiddenState
    ]];

S113BEnvFinalState[
    seq_List
] :=

```

```

Fold[
  S113BEnvStep,
  {1, 2, 3, 4},
  seq
];

S113BEnvRawY[
  seq_List,
  start_,
  target_
] :=
Module[
  {
    hiddenState,
    startIndex,
    targetIndex
  },

  hiddenState =
    S113BEnvFinalState[
      seq
    ];

  startIndex =
    Lookup[
      queryIndexS113B,
      start,
      Missing["UnknownStart"]
    ];

  targetIndex =
    Lookup[
      queryIndexS113B,
      target,
      Missing["UnknownTarget"]
    ];

  If[
    MissingQ[startIndex] ||
    MissingQ[targetIndex],

    Return[
      Missing["UnknownQuery"]
    ]
  ];

  Boole[
    hiddenState[[startIndex]] ===
    targetIndex
  ]

```

```

];

(* ===== *)
(* 4. MEMBERSHIP-QUERY ORACLE + CACHE *)
(* ===== *)

oracleCacheS113B =
  <||>;

oracleUniqueQueryCountS113B =
  0;

oracleTotalCallCountS113B =
  0;

S113BOracleKey[
  seq_List,
  start_,
  target_
] :=
  ToString[
    InputForm[
      {
        seq,
        start,
        target
      }
    ]
  ];

S113BOracleY[
  seq_List,
  start_,
  target_
] :=
  Module[
    {
      key,
      y
    },
    oracleTotalCallCountS113B++;

    key =
      S113BOracleKey[
        seq,
        start,
        target
      ];

```

```

If[
  KeyExistsQ[
    oracleCacheS113B,
    key
  ],

  Return[
    Lookup[
      oracleCacheS113B,
      key
    ]
  ]
];

y =
  S113BEnvRawY[
    seq,
    start,
    target
  ];

If[
  !MemberQ[
    {0, 1},
    y
  ],

  S113BFail[
    "Environment returned a non-binary value."
  ]
];

oracleUniqueQueryCountS113B++;

AssociateTo[
  oracleCacheS113B,
  key -> y
];

y
];

(* ===== *)
(* 5. BEHAVIORAL HYPOTHESIS UNIVERSE *)
(* *)
(* Task-level structural prior: state behavior is a bijection *)
(* of the four query symbols. *)
(* *)

```

```

(* There are 4! possible behavioral hypotheses. *)
(* This does NOT assert that all are reachable. *)
(* It is NOT used as a search stopping condition. *)
(* ===== *)

behaviorHypothesesS113B =
  Permutations[
    querySymbolsS113B
  ];

Print[
  "Task-level behavioral hypotheses = ",
  Length[
    behaviorHypothesesS113B
  ]
];

S113BCandidateY[
  candidate_List,
  start_,
  target_
] :=
Module[
  {
    startIndex
  },

  startIndex =
    Lookup[
      queryIndexS113B,
      start,
      Missing["UnknownStart"]
    ];

  If[
    MissingQ[startIndex],

    Return[
      Missing["UnknownStart"]
    ]
  ];

  Boole[
    candidate[[startIndex]] ==
      target
  ]
];

S113BSignatureFromBijection[
  candidate_List

```



```

] :=
  Table[

    S113BCandidateY[
      candidate,
      pair[[1]],
      pair[[2]]
    ],

    {
      pair,
      allQueryPairsS113B
    }
  ];

S113BBehaviorKey[
  candidate_List
] :=
  ToString[
    InputForm[
      candidate
    ]
  ];

(* ===== *)
(* 6. ADAPTIVE QUERY SELECTION *)
(* *)
(* Among unasked queries, choose the one that minimizes: *)
(* *)
(*   max(#candidate answer 0, #candidate answer 1) *)
(* *)
(* i.e. strongest worst-case split. *)
(* ===== *)

S113BChooseQuery[
  candidates_List,
  askedQueries_List
] :=
  Module[
    {
      available,
      scored,
      informative,
      best
    },

    available =
      Select[
        allQueryPairsS113B,
        !MemberQ[
          askedQueries,
          #
        ] &
      ] &

```

```

];

If[
  Length[available] == 0,

  Return[
    Missing[
      "NoAvailableQuery"
    ]
  ]
];

```

```

scored =
  Table[

    Module[
      {
        answers,
        n0,
        n1
      },

      answers =
        Table[

          S113BCandidateY[
            candidate,
            query[[1]],
            query[[2]]
          ],

          {
            candidate,
            candidates
          }
        ];

      n0 =
        Count[
          answers,
          0
        ];

      n1 =
        Count[
          answers,
          1
        ];

```

<|

```

    "Query" ->
      query,

    "N0" ->
      n0,

    "N1" ->
      n1,

    "WorstRemaining" ->
      Max[
        n0,
        n1
      ],

    "Imbalance" ->
      Abs[
        n0 - n1
      ]

    |>

  ],

  {
    query,
    available
  }
];

informative =
  Select[
    scored,

    And[
      #["N0"] > 0,
      #["N1"] > 0
    ] &
  ];

If[
  Length[informative] == 0,

  Return[
    Missing[
      "NoInformativeQuery"
    ]
  ]
];

best =
  First[

```

```

SortBy[
  informative,

  Function[
    row,

    {
      row["WorstRemaining"],
      row["Imbalance"],
      ToString[
        InputForm[
          row["Query"]
        ]
      ]
    }
  ]
];

best["Query"]
];

(* ===== *)
(* 7. ADAPTIVELY IDENTIFY ONE PREFIX'S BEHAVIOR *)
(* *)
(* Stops immediately when one behavioral hypothesis remains. *)
(* *)
(* NO full 16-bit oracle scan is performed. *)
(* ===== *)

identificationLogS113B =
  {};

identifiedSequenceKeysS113B =
  <||>;

S113BIdentifyBehavior[
  seq_List
] :=
Module[
  {
    candidates,
    asked,
    answers,
    query,
    y,
    beforeCount,
    afterCount,
    result,
    uniqueBefore,
    uniqueAfter,
    seqKey

```

```

},

candidates =
  behaviorHypothesesS113B;

asked =
  {};

answers =
  {};

uniqueBefore =
  oracleUniqueQueryCountS113B;

While[
  Length[candidates] > 1,

  query =
    S113BChooseQuery[
      candidates,
      asked
    ];

  If[
    MissingQ[query],

    S113BFail[
      StringJoin[
        "Unable to distinguish remaining behavioral hypotheses for sequence ",
        ToString[
          InputForm[
            seq
          ]
        ]
      ]
    ]
  ]
];

beforeCount =
  Length[
    candidates
  ];

y =
  S113BOracleY[
    seq,
    query[[1]],

```

```

        query[[2]]
    ];

    AppendTo[
        asked,
        query
    ];

    AppendTo[
        answers,
        y
    ];

    candidates =
        Select[
            candidates,

            S113BCandidateY[
                #,
                query[[1]],
                query[[2]]
            ] ==
            y &
        ];

    afterCount =
        Length[
            candidates
        ];

    If[
        afterCount == 0,

        S113BFail[
            StringJoin[
                "Observed answers are inconsistent with the declared bijection behavior c
                ToString[
                    InputForm[
                        seq
                    ]
                ]
            ]
        ];

    If[
        afterCount >= beforeCount,

        S113BFail[
            "Adaptive query failed to reduce the candidate set."
        ]
    ]

```

```
    ]
  ];

];

result =
  First[
    candidates
  ];

uniqueAfter =
  oracleUniqueQueryCountS113B;

seqKey =
  ToString[
    InputForm[
      seq
    ]
  ];

AssociateTo[
  identifiedSequenceKeysS113B,
  seqKey -> True
];

AppendTo[
  identificationLogS113B,

<|

  "Sequence" ->
    seq,

  "Depth" ->
    Length[seq],

  "QueriesAsked" ->
    Length[asked],

  "UniqueQueriesAdded" ->
    (
      uniqueAfter -
      uniqueBefore
    ),

  "AskedQueries" ->
    asked,

  "Answers" ->
    answers,
```

```

    "IdentifiedBijection" ->
        result

    |>
];

result
];

(* ===== *)
(* 8. ROOT STATE *)
(* ===== *)

behaviorToStateS113B =
    <||>;

stateBijectionS113B =
    <||>;

stateSignatureS113B =
    <||>;

stateRepresentativeSeqS113B =
    <||>;

transitionByStateS113B =
    <||>;

discoveryRowsS113B =
    {};

S113BTransitionKey[
    state_Integer,
    token_Integer
] :=
    ToString[
        InputForm[
            {
                state,
                token
            }
        ]
    ];

rootBehaviorS113B =
    S113BIdentifyBehavior[
        {}
    ]

```



```

];

rootBehaviorKeyS113B =
  S113BBehaviorKey[
    rootBehaviorS113B
  ];

stateCountS113B =
  1;

startStateS113B =
  1;

AssociateTo[
  behaviorToStateS113B,
  rootBehaviorKeyS113B -> 1
];

AssociateTo[
  stateBijectionS113B,
  1 -> rootBehaviorS113B
];

AssociateTo[
  stateSignatureS113B,
  1 ->
    S113BSignatureFromBijection[
      rootBehaviorS113B
    ]
];

AssociateTo[
  stateRepresentativeSeqS113B,
  1 -> {}
];

stateQueueS113B =
  {1};

(* ===== *)
(* 9. ACTIVE TRANSITION-CLOSURE DISCOVERY *)
(* ===== *)

Print["====="];
Print["QUERY-EFFICIENT ACTIVE DISCOVERY"];
Print["====="];

```

```
discoveryTimingS113B =
  AbsoluteTiming[

    While[
      Length[
        stateQueueS113B
      ] > 0,

      currentStateS113B =
        First[
          stateQueueS113B
        ];

      stateQueueS113B =
        Rest[
          stateQueueS113B
        ];

      currentRepresentativeS113B =
        Lookup[
          stateRepresentativeSeqS113B,
          currentStateS113B
        ];

      Do[

        successorSeqS113B =
          Append[
            currentRepresentativeS113B,
            token
          ];

        successorBehaviorS113B =
          S113BIdentifyBehavior[
            successorSeqS113B
          ];

        successorBehaviorKeyS113B =
          S113BBehaviorKey[
            successorBehaviorS113B
          ];

        newStateQ113B =
          !KeyExistsQ[
            behaviorToStateS113B,
            successorBehaviorKeyS113B
          ];
      ]
    ]
  ];
```

```
If[
  newStateQ113B,

  stateCountS113B++;

  successorStateS113B =
    stateCountS113B;

  AssociateTo[
    behaviorToStateS113B,
    successorBehaviorKeyS113B ->
      successorStateS113B
  ];

  AssociateTo[
    stateBijectionS113B,
    successorStateS113B ->
      successorBehaviorS113B
  ];

  AssociateTo[
    stateSignatureS113B,
    successorStateS113B ->
      S113BSignatureFromBijection[
        successorBehaviorS113B
      ]
  ];

  AssociateTo[
    stateRepresentativeSeqS113B,
    successorStateS113B ->
      successorSeqS113B
  ];

  AppendTo[
    stateQueueS113B,
    successorStateS113B
  ],

  successorStateS113B =
    Lookup[
      behaviorToStateS113B,
      successorBehaviorKeyS113B
    ]
];
```

```

AssociateTo[
  transitionByStateS113B,

  S113BTransitionKey[
    currentStateS113B,
    token
  ] ->
  successorStateS113B
];

AppendTo[
  discoveryRowsS113B,

  <|

  "SourceState" ->
    currentStateS113B,

  "Token" ->
    token,

  "DestinationState" ->
    successorStateS113B,

  "SuccessorSeq" ->
    successorSeqS113B,

  "CreatedNewState" ->
    newStateQ113B

  |>
];

,
{
  token,
  alphabetS113B
}
];

(* safety guard only *)

If[
  stateCountS113B > 4096,

  S113BFail[
    "State-space safety guard exceeded 4096."
  ]
];

];

];

```

```

Print[
  "Discovery seconds = ",
  N[
    discoveryTimingS113B[[1]]
  ]
];

Print[
  "Observable states discovered = ",
  stateCountS113B
];

Print[
  "Transitions discovered = ",
  Length[
    transitionByStateS113B
  ]
];

(* ===== *)
(* 10. QUERY-EFFICIENCY STATISTICS *)
(* ===== *)

uniqueIdentifiedSequencesS113B =
  Length[
    identifiedSequenceKeysS113B
  ];

fullSignatureBaselineQueriesS113B =
  uniqueIdentifiedSequencesS113B *
  Length[
    allQueryPairsS113B
  ];

membershipQueriesBeforeFreezeS113B =
  oracleUniqueQueryCountS113B;

querySavingsS113B =
  fullSignatureBaselineQueriesS113B -
  membershipQueriesBeforeFreezeS113B;

querySavingsFractionS113B =
  N[
    querySavingsS113B /
    fullSignatureBaselineQueriesS113B
  ];

```

```
queriesPerIdentificationS113B =  
  (  
    #["QueriesAsked"] &  
  ) /@  
    identificationLogS113B;  
  
meanQueriesPerIdentificationS113B =  
  N[  
    Mean[  
      queriesPerIdentificationS113B  
    ]  
  ];  
  
maxQueriesPerIdentificationS113B =  
  Max[  
    queriesPerIdentificationS113B  
  ];  
  
minQueriesPerIdentificationS113B =  
  Min[  
    queriesPerIdentificationS113B  
  ];  
  
queryCountHistogramS113B =  
  Counts[  
    queriesPerIdentificationS113B  
  ];  
  
Print[  
  "Unique identified sequences = ",  
  uniqueIdentifiedSequencesS113B  
];  
  
Print[  
  "Full-16-query baseline = ",  
  fullSignatureBaselineQueriesS113B  
];  
  
Print[  
  "Actual membership queries = ",  
  membershipQueriesBeforeFreezeS113B  
];  
  
Print[  
  "Queries saved = ",  
  querySavingsS113B  
];
```

```

Print[
  "Query savings fraction = ",
  querySavingsFractionS113B
];

Print[
  "Mean queries / identification = ",
  meanQueriesPerIdentificationS113B
];

Print[
  "Min / Max queries = ",
  minQueriesPerIdentificationS113B,
  " / ",
  maxQueriesPerIdentificationS113B
];

Print[
  "Query-count histogram = ",
  queryCountHistogramS113B
];

(* ===== *)
(* 11. STATE / TRANSITION AUDITS BEFORE FREEZE *)
(* ===== *)

stateBehaviorAuditS113B =
  AllTrue[
    Values[
      stateBijectionS113B
    ],
    Function[
      bij,

      And[
        ListQ[bij],
        Length[bij] == 4,
        Sort[bij] ==
          querySymbolsS113B
      ]
    ]
  ];

distinctBehaviorCountS113B =
  Length[
    DeleteDuplicates[
      Values[
        stateBijectionS113B
      ]
    ]
  ]

```

```
    ]
];

distinctSignatureCountS113B =
  Length[
    DeleteDuplicates[
      Values[
        stateSignatureS113B
      ]
    ]
  ];

expectedTransitionCountS113B =
  stateCountS113B *
  Length[
    alphabetS113B
  ];

transitionCompletenessS113B =
  N[
    Length[
      transitionByStateS113B
    ] /
    expectedTransitionCountS113B
  ];

Print[
  "State behavior audit = ",
  stateBehaviorAuditS113B
];

Print[
  "Distinct behavioral states = ",
  distinctBehaviorCountS113B
];

Print[
  "Distinct derived signatures = ",
  distinctSignatureCountS113B
];

Print[
  "Transitions = ",
  Length[
    transitionByStateS113B
  ],
  " / ",
  expectedTransitionCountS113B
];
```



```

Print[
  "Transition completeness = ",
  transitionCompletenessS113B
];

If[
  !TrueQ[stateBehaviorAuditS113B],

  S113BFail[
    "Discovered state has invalid bijection behavior."
  ]
];

If[
  Abs[
    transitionCompletenessS113B - 1.
  ] > 10^-12,

  S113BFail[
    "Transition table is incomplete."
  ]
];

(* ===== *)
(* 12. FROZEN MACHINE EXECUTION *)
(* ===== *)

S113BExecuteState[
  seq_List
] :=
Module[
  {
    state
  },

  state =
    startStateS113B;

  Do[

    state =
      Lookup[
        transitionByStateS113B,

        S113BTransitionKey[
          state,
          token
        ],

```

```

        Missing[
            "UnknownTransition"
        ]
    ];

    If[
        MissingQ[state],
        Return[state]
    ];

    ,
    {
        token,
        seq
    }
];

state
];

S113BLearnedSignature[
    seq_List
] :=
Module[
    {
        state
    },

    state =
        S113BExecuteState[
            seq
        ];

    If[
        MissingQ[state],

        Return[
            Missing[
                "UnknownTransition"
            ]
        ]
    ];

    Lookup[
        stateSignatureS113B,
        state,
        Missing[
            "UnknownState"
        ]
    ]
];

```

```

];

S113BPredict[
  seq_List,
  start_,
  target_
] :=
Module[
  {
    state,
    behavior,
    startIndex
  },

  state =
    S113BExecuteState[
      seq
    ];

  If[
    MissingQ[state],

    Return[
      Missing[
        "UnknownTransition"
      ]
    ]
  ];

  behavior =
    Lookup[
      stateBijectionS113B,
      state,
      Missing[
        "UnknownState"
      ]
    ];

  startIndex =
    Lookup[
      queryIndexS113B,
      start,
      Missing[
        "UnknownStart"
      ]
    ];

  If[
    MissingQ[behavior] ||
    MissingQ[startIndex],

```

```

Return[
  Missing[
    "Unavailable"
  ]
];

Boole[
  behavior[[startIndex]] ==
  target
];

(* ===== *)
(* 13. HOLDOUT LEAKAGE CHECK *)
(* *)
(* Cache inspection only. No new queries. *)
(* ===== *)

S113BHoldoutTouchedQ[
  seq_List
] :=
AnyTrue[
  allQueryPairsS113B,

  Function[
    pair,

    KeyExistsQ[
      oracleCacheS113B,

      S113BOracleKey[
        seq,
        pair[[1]],
        pair[[2]]
      ]
    ]
  ]
];

holdoutTouchedBeforeFreezeS113B =
Count[
  prospectiveHoldoutSeqsS113B,

  row_Association /;
  S113BHoldoutTouchedQ[
    row["Sequence"]
  ]
];

Print[

```

```

"Holdout sequences touched before freeze = ",
holdoutTouchedBeforeFreezeS113B,
" / ",
Length[
    prospectiveHoldoutSeqsS113B
]
];

(* ===== *)
(* 14. FREEZE *)
(* ===== *)

machineHashS113B =
Hash[
{
    stateBijectionS113B,
    stateSignatureS113B,
    stateRepresentativeSeqS113B,
    transitionByStateS113B,
    startStateS113B
},
    "SHA256",
    "HexString"
];

Print["====="];
Print["S113B MACHINE FROZEN"];

Print[
    "States = ",
    stateCountS113B
];

Print[
    "Transitions = ",
    Length[
        transitionByStateS113B
    ]
];

Print[
    "Membership queries = ",
    membershipQueriesBeforeFreezeS113B
];

Print[
    "Full-signature baseline = ",
    fullSignatureBaselineQueriesS113B
];

```

```

Print[
  "Query savings = ",
  100. querySavingsFractionS113B,
  "%"
];

Print[
  "Machine hash = ",
  machineHashS113B
];

Print["NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT."];
Print["====="];

(* ===== *)
(* 15. POST-FREEZE LONG HOLDOUT *)
(* ===== *)

Print["====="];
Print["OPENING LONG HOLDOUT AFTER FREEZE"];
Print["====="];

holdoutRowsS113B =
  Table[
    Module[
      {
        seq,
        learnedSignature,
        oracleSignature
      },
      seq =
        row[
          "Sequence"
        ];

      learnedSignature =
        S113BLearnedSignature[
          seq
        ];

      oracleSignature =
        Table[
          S113BOracleY[
            seq,

```

```

        pair[[1]],
        pair[[2]]
    ],

    {
        pair,
        allQueryPairsS113B
    }
];

<|

"Depth" ->
    row["Depth"],

"LearnedState" ->
    S113BExecuteState[
        seq
    ],

"SignatureMatch" ->
    (
        learnedSignature ===
        oracleSignature
    )

|>

],

{
    row,
    prospectiveHoldoutSeqsS113B
}
];

holdoutMatchesS113B =
    Count[
        (
            TrueQ[
                #["SignatureMatch"]
            ] &
        ) /@
        holdoutRowsS113B,
        True
    ];

holdoutAccuracyS113B =
    N[
        holdoutMatchesS113B /
        Length[
            holdoutRowsS113B
        ]
    ]

```

```

];

holdoutPerDepthS113B =
  Table[

    Module[
      {
        rows,
        matches
      },

      rows =
        Select[
          holdoutRowsS113B,
          #["Depth"] === depth &
        ];

      matches =
        Count[
          (
            TrueQ[
              #["SignatureMatch"]
            ] &
          ) /@
            rows,
          True
        ];

      <|

        "Depth" ->
          depth,

        "Sequences" ->
          Length[
            rows
          ],

        "Matches" ->
          matches,

        "Accuracy" ->
          N[
            matches /
              Length[
                rows
              ]
          ]

      |>

    ],

```



```

    {
      depth,
      holdoutDepthsS113B
    }
  ];

Print[
  "Long holdout = ",
  holdoutMatchesS113B,
  " / ",
  Length[
    holdoutRowsS113B
  ]
];

Print[
  "Long holdout accuracy = ",
  holdoutAccuracyS113B
];

(* ===== *)
(* 16. POST-FREEZE EXHAUSTIVE LENGTH 0.6 AUDIT *)
(* ===== *)

auditMaxLengthS113B =
  6;

auditSequencesS113B =
  Flatten[
    Table[
      Tuples[
        alphabetS113B,
        len
      ],
      {
        len,
        0,
        auditMaxLengthS113B
      }
    ],
    1
  ];

auditRowsS113B =
  Table[
    Module[

```

```

    {
        learnedSignature,
        oracleSignature
    },

    learnedSignature =
        S113BLearnedSignature[
            seq
        ];

    oracleSignature =
        Table[

            S113BOracleY[
                seq,
                pair[[1]],
                pair[[2]]
            ],

            {
                pair,
                allQueryPairsS113B
            }
        ];

<|

    "Length" ->
        Length[
            seq
        ],

    "SignatureMatch" ->
        (
            learnedSignature ===
            oracleSignature
        )

|>

],

{
    seq,
    auditSequencesS113B
}
];

auditMatchesS113B =
    Count[
        (
            TrueQ[

```

```

        #["SignatureMatch"]
    ] &
) /@
    auditRowsS113B,
    True
];

auditAccuracyS113B =
N[
    auditMatchesS113B /
    Length[
        auditRowsS113B
    ]
];

auditPerLengthS113B =
Table[

    Module[
        {
            rows,
            matches
        },

        rows =
        Select[
            auditRowsS113B,
            #["Length"] === len &
        ];

        matches =
        Count[
            (
                TrueQ[
                    #["SignatureMatch"]
                ] &
            ) /@
            rows,
            True
        ];

        <|

            "Length" ->
                len,

            "Sequences" ->
                Length[
                    rows
                ],

```

```

    "Matches" ->
        matches,

    "Accuracy" ->
        N[
            matches /
            Length[
                rows
            ]
        ]

    |>

],

{
    len,
    0,
    auditMaxLengthS113B
}
];

Print[
    "Exhaustive length 0..",
    auditMaxLengthS113B,
    " = ",
    auditMatchesS113B,
    " / ",
    Length[
        auditRowsS113B
    ]
];

(* ===== *)
(* 17. VERIFY FROZEN HASH BEFORE TRUE-S4 AUDIT *)
(* ===== *)

recomputedHashS113B =
    Hash[
        {
            stateBijectionS113B,
            stateSignatureS113B,
            stateRepresentativeSeqS113B,
            transitionByStateS113B,
            startStateS113B
        },
        "SHA256",
        "HexString"
    ];

frozenHashMatchesS113B =
    recomputedHashS113B ==

```

```

machineHashS113B;

Print[
  "Frozen hash unchanged = ",
  frozenHashMatchesS113B
];

If[
  !TrueQ[
    frozenHashMatchesS113B
  ],

  S113BFail[
    "Machine changed after freeze."
  ]
];

(* ===== *)
(* 18. POST-HOC TRUE S4 STATE SPACE *)
(* ===== *)

Print["====="];
Print["POST-HOC TRUE S4 ISOMORPHISM AUDIT"];
Print["====="];

trueStatesS113B =
  Permutations[
    {1, 2, 3, 4}
  ];

trueStateCountS113B =
  Length[
    trueStatesS113B
  ];

trueStartStateS113B =
  {1, 2, 3, 4};

S113BTrueQuery[
  hiddenState_List,
  start_,
  target_
] :=
Module[
  {
    startIndex,
    targetIndex
  },

```

```

startIndex =
  Lookup[
    queryIndexS113B,
    start
  ];

targetIndex =
  Lookup[
    queryIndexS113B,
    target
  ];

Boole[
  hiddenState[[startIndex]] ==
  targetIndex
];

S113BTrueSignature[
  hiddenState_List
] :=
  Table[

    S113BTrueQuery[
      hiddenState,
      pair[[1]],
      pair[[2]]
    ],

    {
      pair,
      allQueryPairsS113B
    }
  ];

trueDistinctSignaturesS113B =
  Length[
    DeleteDuplicates[
      S113BTrueSignature /@
      trueStatesS113B
    ]
  ];

Print[
  "True S4 states = ",
  trueStateCountS113B
];

Print[

```

```

"Distinct true signatures = ",
trueDistinctSignaturesS113B
];

(* ===== *)
(* 19. LEARNED -> TRUE STATE CORRESPONDENCE *)
(* ===== *)

stateCorrespondencesS113B =
  Table[
    Module[
      {
        learnedSignature,
        matches
      },

      learnedSignature =
        Lookup[
          stateSignatureS113B,
          learnedState
        ];

      matches =
        Select[
          trueStatesS113B,

          S113BTrueSignature[#] ===
            learnedSignature &
        ];

      <|

        "LearnedState" ->
          learnedState,

        "RepresentativeSeq" ->
          Lookup[
            stateRepresentativeSeqS113B,
            learnedState
          ],

        "LearnedBijection" ->
          Lookup[
            stateBijectionS113B,
            learnedState
          ],

        "TrueMatches" ->
          matches,

        "MatchCount" ->

```

```

        Length[
            matches
        ],

        "Functional" ->
        (
            Length[
                matches
            ] == 1
        )

        |>

    ],

    {
        learnedState,
        Range[
            stateCountS113B
        ]
    }
];

learnedToTrueFunctionalS113B =
    AllTrue[
        stateCorrespondenceS113B,

        Function[
            row,
            TrueQ[
                row["Functional"]
            ]
        ]
    ];

assignedTrueStatesS113B =
    If[
        learnedToTrueFunctionalS113B,

        (
            First[
                #["TrueMatches"]
            ] &
        ) /@
        stateCorrespondenceS113B,

        {}
    ];

trueToLearnedFunctionalS113B =
    If[
        learnedToTrueFunctionalS113B,

```



```

AllTrue[
  trueStatesS113B,

  Function[
    trueState,

    Count[
      assignedTrueStatesS113B,
      trueState
    ] == 1
  ]
],

False
];

stateBijectionPassedS113B =
And[
  learnedToTrueFunctionalS113B,
  trueToLearnedFunctionalS113B,
  stateCountS113B ==
    trueStateCountS113B
];

Print[
  "Learned -> true functional = ",
  learnedToTrueFunctionalS113B
];

Print[
  "True -> learned functional = ",
  trueToLearnedFunctionalS113B
];

Print[
  "State-space bijection = ",
  stateBijectionPassedS113B
];

phiS113B =
If[
  stateBijectionPassedS113B,

  Association[
    Table[

      row["LearnedState"] ->
        First[
          row["TrueMatches"]
        ],
    ],
  ],
];

```

```

        {
            row,
            stateCorrespondenceS113B
        }
    ],
    <||>
];

startStateMatchS113B =
    If[
        stateBijectionPassedS113B,

        Lookup[
            phiS113B,
            startStateS113B,
            Missing["NoMapping"]
        ] ==
            trueStartStateS113B,

        False
    ];

Print[
    "Start-state correspondence = ",
    startStateMatchS113B
];

(* ===== *)
(* 20. COMPLETE TRANSITION ISOMORPHISM *)
(* ===== *)

S113BLearnedStep[
    learnedState_Integer,
    token_Integer
] :=
    Lookup[
        transitionByStateS113B,

        S113BTransitionKey[
            learnedState,
            token
        ],

        Missing[
            "UnknownTransition"
        ]
    ];

transitionAuditRowsS113B =
    Flatten[

```

```

Table[

  Module[
    {
      learnedDestination,
      trueSource,
      mappedLearnedDestination,
      trueDestination
    },

    learnedDestination =
      S113BLearnedStep[
        learnedState,
        token
      ];

    trueSource =
      Lookup[
        phiS113B,
        learnedState,
        Missing[
          "NoPhi"
        ]
      ];

    mappedLearnedDestination =
      If[
        IntegerQ[
          learnedDestination
        ],
        Lookup[
          phiS113B,
          learnedDestination,
          Missing[
            "NoDestPhi"
          ]
        ],
        Missing[
          "NoLearnedDestination"
        ]
      ];

    trueDestination =
      If[
        ListQ[
          trueSource
        ],
        S113BEnvStep[
          trueSource,

```

```
        token
      ],

      Missing[
        "NoTrueSource"
      ]
    ];

<|

  "LearnedSource" ->
    learnedState,

  "Token" ->
    token,

  "LearnedDestination" ->
    learnedDestination,

  "TrueSource" ->
    trueSource,

  "MappedLearnedDestination" ->
    mappedLearnedDestination,

  "TrueDestination" ->
    trueDestination,

  "Match" ->
    (
      mappedLearnedDestination ===
      trueDestination
    )

|>

],

{
  learnedState,
  Range[
    stateCountS113B
  ]
},

{
  token,
  alphabetS113B
}
],

1
];
```

```

transitionTotalS113B =
  Length[
    transitionAuditRowsS113B
  ];

transitionMatchesS113B =
  Count[
    (
      TrueQ[
        #["Match"]
      ] &
    ) /@
    transitionAuditRowsS113B,
    True
  ];

transitionIsomorphismPassedS113B =
  transitionMatchesS113B ==
    transitionTotalS113B;

Print[
  "Transition correspondence = ",
  transitionMatchesS113B,
  " / ",
  transitionTotalS113B
];

(* ===== *)
(* 21. COMPLETE STATE x QUERY AUDIT *)
(* ===== *)

queryAuditRowsS113B =
  Flatten[
    Table[

      Module[
        {
          trueState
        },

        trueState =
          Lookup[
            phiS113B,
            learnedState,
            Missing[
              "NoPhi"
            ]
          ]
      ];

    Table[

```

```

<|

"LearnedState" ->
  learnedState,

"Start" ->
  pair[[1]],

"Target" ->
  pair[[2]],

"LearnedY" ->
  S113BCandidateY[
    Lookup[
      stateBijectionS113B,
      learnedState
    ],
    pair[[1]],
    pair[[2]]
  ],

"TrueY" ->
  If[
    ListQ[
      trueState
    ],

    S113BTrueQuery[
      trueState,
      pair[[1]],
      pair[[2]]
    ],

    Missing[
      "NoTrueState"
    ]
  ],

"Match" ->
  (
    S113BCandidateY[
      Lookup[
        stateBijectionS113B,
        learnedState
      ],
      pair[[1]],
      pair[[2]]
    ] ==
    If[
      ListQ[
        trueState
      ],

      S113BTrueQuery[

```

```

        trueState,
        pair[[1]],
        pair[[2]]
    ],
    Missing[
        "NoTrueState"
    ]
]
)

|>,

{
    pair,
    allQueryPairsS113B
}
]

],

{
    learnedState,
    Range[
        stateCountS113B
    ]
}
],

1
];

queryTotalS113B =
    Length[
        queryAuditRowsS113B
    ];

queryMatchesS113B =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        queryAuditRowsS113B,
        True
    ];

fullQueryBehaviorPassedS113B =
    queryMatchesS113B ==
        queryTotalS113B;

```

```

Print[
  "Complete state/query behavior = ",
  queryMatchesS113B,
  " / ",
  queryTotalS113B
];

(* ===== *)
(* 22. SYNCHRONIZED PRODUCT CLOSURE *)
(* ===== *)

S113BProductClosure[] :=
Module[
  {
    startPair,
    reached,
    frontier,
    nextPairs,
    newPairs,
    learnedNext,
    trueNext,
    guard
  },

  startPair =
  {
    startStateS113B,
    trueStartStateS113B
  };

  reached =
  {
    startPair
  };

  frontier =
  {
    startPair
  };

  guard =
  0;

  While[
    Length[
      frontier
    ] > 0,

    guard++;

```



```

If[
  guard > 1000,

  Return[$Failed]
];

nextPairs =
  DeleteDuplicates[
    Flatten[
      Table[

        learnedNext =
          S113BLearnedStep[
            pair[[1]],
            token
          ];

        trueNext =
          S113BEnvStep[
            pair[[2]],
            token
          ];

        {
          learnedNext,
          trueNext
        },

        {
          pair,
          frontier
        },

        {
          token,
          alphabetS113B
        }
      ],

      1
    ]
  ];

If[
  AnyTrue[
    nextPairs,

    Function[
      pair,
      !IntegerQ[

```

```

        pair[[1]]
      ]
    ],
    Return[$Failed]
  ];

  newPairs =
    Select[
      nextPairs,

      Function[
        pair,
        !MemberQ[
          reached,
          pair
        ]
      ]
    ]
  ];

  reached =
    Join[
      reached,
      newPairs
    ];

  frontier =
    newPairs;

];

reached
];

productPairsS113B =
  S113BProductClosure[];

If[
  productPairsS113B === $Failed,

  S113BFail[
    "Synchronized product exploration failed."
  ]
];

productPairCountS113B =
  Length[
    productPairsS113B
  ]

```

```

];

productLearnedFunctionalS113B =
  AllTrue[
    Range[
      stateCountS113B
    ],
    Function[
      learnedState,

      Length[
        DeleteDuplicates[
          (
            #[[2]] &
          ) /@
          Select[
            productPairsS113B,
            #[[1]] ===
              learnedState &
          ]
        ]
      ] == 1
    ]
  ];

```

```

productTrueFunctionalS113B =
  AllTrue[
    trueStatesS113B,

    Function[
      trueState,

      Length[
        DeleteDuplicates[
          (
            #[[1]] &
          ) /@
          Select[
            productPairsS113B,
            #[[2]] ===
              trueState &
          ]
        ]
      ] == 1
    ]
  ];

```

```

productBijectionPassedS113B =
  And[
    productPairCountS113B ==
      stateCountS113B,

```

```

    productLearnedFunctionalS113B,

    productTrueFunctionalS113B
];

Print[
  "Synchronized product pairs = ",
  productPairCountS113B
];

Print[
  "Product learned->true functional = ",
  productLearnedFunctionalS113B
];

Print[
  "Product true->learned functional = ",
  productTrueFunctionalS113B
];

(* ===== *)
(* 23. DIRECT BEHAVIOR ENCODING *)
(* ===== *)

directEncodingRowsS113B =
  Table[

    Module[
      {
        learnedBehavior,
        mappedTrue
      },

      learnedBehavior =
        Lookup[
          stateBijectionS113B,
          learnedState
        ];

      mappedTrue =
        Lookup[
          phiS113B,
          learnedState,
          Missing[
            "NoPhi"
          ]
        ];
    ],
  ];

<|

```

```

    "LearnedState" ->
        learnedState,

    "LearnedBijection0123" ->
        learnedBehavior,

    "Converted1234" ->
        (
            learnedBehavior + 1
        ),

    "MappedTrueS4" ->
        mappedTrue,

    "Match" ->
        (
            learnedBehavior + 1 ==
            mappedTrue
        )

    |>

],

{
    learnedState,
    Range[
        stateCountS113B
    ]
}
];

directEncodingMatchesS113B =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        directEncodingRowsS113B,
        True
    ];

directEncodingExactS113B =
    directEncodingMatchesS113B ==
    stateCountS113B;

Print[
    "Direct behavioral encoding = ",
    directEncodingMatchesS113B,
    " / ",
    stateCountS113B

```

```

];

(* ===== *)
(* 24. EXACTNESS + EFFICIENCY GATES *)
(* ===== *)

exactMachineIsomorphismS113B =
  And[
    frozenHashMatchesS113B,
    stateBijectionPassedS113B,
    startStateMatchS113B,
    transitionIsomorphismPassedS113B,
    fullQueryBehaviorPassedS113B,
    productBijectionPassedS113B
  ];

allFiniteSequenceEquivalentS113B =
  exactMachineIsomorphismS113B;

stateCountCorrectPostHocS113B =
  stateCountS113B ==
    trueStateCountS113B;

queryEfficientS113B =
  membershipQueriesBeforeFreezeS113B <
    fullSignatureBaselineQueriesS113B;

strongQueryReductionS113B =
  querySavingsFractionS113B >=
    0.25;

strongSuccessS113B =
  And[
    holdoutTouchedBeforeFreezeS113B == 0,
    stateCountCorrectPostHocS113B,
    Abs[
      holdoutAccuracyS113B - 1.
    ] < 10^-12,
    Abs[
      auditAccuracyS113B - 1.
    ] < 10^-12,
    exactMachineIsomorphismS113B,
    queryEfficientS113B
  ];

diagnosisS113B =
  Which[

    strongSuccessS113B &&

```

```

strongQueryReductionS113B &&
directEncodingExactS113B,

    "QUERY_EFFICIENT_S4_SUCCESS_EXACT_24_STATE_MACHINE_RECOVERED_WITH_STRONG_MEMB

strongSuccessS113B &&
directEncodingExactS113B,

    "QUERY_EFFICIENT_S4_SUCCESS_EXACT_MACHINE_RECOVERED_WITH_FEWER_QUERIES_BUT_RE

!stateCountCorrectPostHocS113B,

    "QUERY_EFFICIENT_POLICY_FAILS_TO_RECOVER_THE_COMPLETE_S4_STATE_SPACE",

!exactMachineIsomorphismS113B,

    "QUERY_EFFICIENT_POLICY_FINDS_A_MACHINE_BUT_IT_IS_NOT_EXACTLY_ISOMORPHIC_TO_S

!queryEfficientS113B,

    "MACHINE_IS_EXACT_BUT_QUERY_POLICY_DOES_NOT_BEAT_FULL_SIGNATURE_BASELINE",

holdoutAccuracyS113B < 1.,

    "QUERY_EFFICIENT_MACHINE_FAILS_LONG_SEQUENCE_HOLDOUT",

True,

    "INCONCLUSIVE"

];

(* ===== *)
(* 25. STATE TABLE *)
(* ===== *)

stateTableS113B =
  Table[

    <|

      "State" ->
        state,

      "RepresentativeSeq" ->
        Lookup[
          stateRepresentativeSeqS113B,
          state
        ],

    ]

```

```

"RepresentativeLength" ->
  Length[
    Lookup[
      stateRepresentativeSeqS113B,
      state
    ]
  ],

"Behavior" ->
  Lookup[
    stateBijectionS113B,
    state
  ],

"On0" ->
  Lookup[
    transitionByStateS113B,
    S113BTransitionKey[
      state,
      0
    ]
  ],

"On1" ->
  Lookup[
    transitionByStateS113B,
    S113BTransitionKey[
      state,
      1
    ]
  ],

"On2" ->
  Lookup[
    transitionByStateS113B,
    S113BTransitionKey[
      state,
      2
    ]
  ],

"TrueS4PostHoc" ->
  Lookup[
    phiS113B,
    state,
    Missing[
      "NoMapping"
    ]
  ]

|>,

{
  state,
  Range[

```



```

        stateCountS113B
    ]
}
];

(* ===== *)
(* 26. FINAL REPORT *)
(* ===== *)

finalS113B =
<|

  "Stage" ->
    "S113B_QueryEfficientActiveS4StateDiscovery",

  "FullSignatureQueriedDuringDiscovery" ->
    False,

  "TaskLevelBijectionPriorUsed" ->
    True,

  "HiddenReachableStateCountUsedForStopping" ->
    False,

  "LearnerReceivesIntermediateState" ->
    False,

  "LearnerReceivesFinalBinaryYOnly" ->
    True,

  "States" ->
    stateCountS113B,

  "DistinctBehaviors" ->
    distinctBehaviorCountS113B,

  "Transitions" ->
    Length[
      transitionByStateS113B
    ],

  "TransitionCompleteness" ->
    transitionCompletenessS113B,

  "UniqueIdentifiedSequences" ->
    uniqueIdentifiedSequencesS113B,

  "FullSignatureBaselineQueries" ->
    fullSignatureBaselineQueriesS113B,

  "ActualMembershipQueries" ->
    membershipQueriesBeforeFreezeS113B,

  "QueriesSaved" ->
    querySavingsS113B,

```

```
"QuerySavingsFraction" ->
  querySavingsFractionS113B,

"MeanQueriesPerIdentification" ->
  meanQueriesPerIdentificationS113B,

"MinQueriesPerIdentification" ->
  minQueriesPerIdentificationS113B,

"MaxQueriesPerIdentification" ->
  maxQueriesPerIdentificationS113B,

"QueryCountHistogram" ->
  queryCountHistogramS113B,

"HoldoutTouchedBeforeFreeze" ->
  holdoutTouchedBeforeFreezeS113B,

"HoldoutSequences" ->
  Length[
    holdoutRowsS113B
  ],

"HoldoutMatches" ->
  holdoutMatchesS113B,

"HoldoutAccuracy" ->
  holdoutAccuracyS113B,

"ExhaustiveSequences" ->
  Length[
    auditRowsS113B
  ],

"ExhaustiveMatches" ->
  auditMatchesS113B,

"ExhaustiveAccuracy" ->
  auditAccuracyS113B,

"StateBijectionPassed" ->
  stateBijectionPassedS113B,

"StartStateMatch" ->
  startStateMatchS113B,

"TransitionMatches" ->
  transitionMatchesS113B,

"TransitionTotal" ->
  transitionTotalS113B,

"QueryBehaviorMatches" ->
  queryMatchesS113B,
```

```

"QueryBehaviorTotal" ->
    queryTotalS113B,

"ProductPairs" ->
    productPairCountS113B,

"ProductBijectionPassed" ->
    productBijectionPassedS113B,

"DirectEncodingMatches" ->
    directEncodingMatchesS113B,

"DirectEncodingExact" ->
    directEncodingExactS113B,

"ExactMachineIsomorphism" ->
    exactMachineIsomorphismS113B,

"AllFiniteSequenceEquivalent" ->
    allFiniteSequenceEquivalentS113B,

"QueryEfficient" ->
    queryEfficientS113B,

"StrongQueryReduction" ->
    strongQueryReductionS113B,

"StrongSuccess" ->
    strongSuccessS113B,

"Diagnosis" ->
    diagnosisS113B,

"MachineHash" ->
    machineHashS113B

|>;

Print["====="];
Print["S113B COMPLETE"];

Print[
    "States = ",
    stateCountS113B
];

Print[
    "Transitions = ",
    Length[
        transitionByStateS113B
    ],
    " / ",
    expectedTransitionCountS113B
];

```

```
];

Print[
  "Full-signature query baseline = ",
  fullSignatureBaselineQueriesS113B
];

Print[
  "Actual membership queries = ",
  membershipQueriesBeforeFreezeS113B
];

Print[
  "Queries saved = ",
  querySavingsS113B,
  " (",
  100. querySavingsFractionS113B,
  "%)"
];

Print[
  "Mean queries / prefix identification = ",
  meanQueriesPerIdentificationS113B
];

Print[
  "Long holdout = ",
  holdoutMatchesS113B,
  " / ",
  Length[
    holdoutRowsS113B
  ]
];

Print[
  "Exhaustive audit = ",
  auditMatchesS113B,
  " / ",
  Length[
    auditRowsS113B
  ]
];

Print[
  "Post-hoc state bijection = ",
  stateBijectionPassedS113B
];
```

```
Print[
  "Transition correspondence = ",
  transitionMatchesS113B,
  " / ",
  transitionTotalS113B
];

Print[
  "Query behavior correspondence = ",
  queryMatchesS113B,
  " / ",
  queryTotalS113B
];

Print[
  "Exact machine isomorphism = ",
  exactMachineIsomorphismS113B
];

Print[
  "All finite-sequence equivalent = ",
  allFiniteSequenceEquivalentS113B
];

Print[
  "Diagnosis = ",
  diagnosisS113B
];

Print["====="];

finalS113B

),

"S113B_STOP"
];
```

```

=====
S113B QUERY-EFFICIENT ACTIVE S4 STATE DISCOVERY
=====
Primitive alphabet = {0, 1, 2}
Query symbols = {0, 1, 2, 3}
Possible binary queries per complete signature = 16
DISCOVERY WILL NOT QUERY ALL 16 BITS BY DEFAULT.
LEARNER IS NOT GIVEN THE HIDDEN REACHABLE-STATE COUNT.
LEARNER RECEIVES FINAL BINARY Y ONLY.
TRUE S4 IS POST-HOC ONLY.
Holdout sequences created = 400
Holdout depths = {12, 20, 40, 80}
HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED.
Task-level behavioral hypotheses = 24
=====
QUERY-EFFICIENT ACTIVE DISCOVERY
=====
Discovery seconds = 1.20904
Observable states discovered = 24
Transitions discovered = 72
Unique identified sequences = 73
Full-16-query baseline = 1168
Actual membership queries = 357
Queries saved = 811
Query savings fraction = 0.694349
Mean queries / identification = 4.89041
Min / Max queries = 3 / 6
Query-count histogram = <|3 -> 7, 4 -> 18, 5 -> 24, 6 -> 24|>
State behavior audit = True
Distinct behavioral states = 24
Distinct derived signatures = 24
Transitions = 72 / 72
Transition completeness = 1.
Holdout sequences touched before freeze = 0 / 400
=====
S113B MACHINE FROZEN
States = 24
Transitions = 72
Membership queries = 357
Full-signature baseline = 1168
Query savings = 69.4349%
Machine hash = 7888756ff993f4fc6a9e3dd895db6426cc0da41dea3c03a926f7c088a129cf\

> 42
NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT.
=====
=====
OPENING LONG HOLDOUT AFTER FREEZE
=====
Long holdout = 400 / 400
Long holdout accuracy = 1.
Exhaustive length 0..6 = 1093 / 1093
Frozen hash unchanged = True
=====
POST-HOC TRUE S4 ISOMORPHISM AUDIT
=====

```

```

True S4 states = 24
Distinct true signatures = 24
Learned -> true functional = True
True -> learned functional = True
State-space bijection = True
Start-state correspondence = True
Transition correspondence = 72 / 72
Complete state/query behavior = 384 / 384
Synchronized product pairs = 24
Product learned->true functional = True
Product true->learned functional = True
Direct behavioral encoding = 24 / 24
=====
S113B COMPLETE
States = 24
Transitions = 72 / 72
Full-signature query baseline = 1168
Actual membership queries = 357
Queries saved = 811 (69.4349%)
Mean queries / prefix identification = 4.89041
Long holdout = 400 / 400
Exhaustive audit = 1093 / 1093
Post-hoc state bijection = True
Transition correspondence = 72 / 72
Query behavior correspondence = 384 / 384
Exact machine isomorphism = True
All finite-sequence equivalent = True
Diagnosis = QUERY_EFFICIENT_S4_SUCCESS_EXACT_24_STATE_MACHINE_RECOVERED_WITH_\

> STRONG_MEMBERSHIP_QUERY_REDUCTION_AND_DIRECT_BEHAVIORAL_ENCODING
=====

```

In [442...

```

(* ===== *)
(* S114A *)
(* FROZEN-POLICY PROSPECTIVE S5 ACTIVE STATE DISCOVERY *)
(* *)
(* POLICY FROZEN FROM S113B *)
(* ----- *)
(* The adaptive query policy is unchanged: *)
(* *)
(* choose q minimizing max(N0(q), N1(q)) *)
(* *)
(* State discovery is unchanged: *)
(* *)
(* - identify observable behavior of a representative prefix *)
(* - reuse an existing state iff behavior is identical *)
(* - otherwise create a new state *)
(* - expand primitive transitions *)
(* - stop ONLY when transition closure is reached *)
(* *)
(* NEW PROSPECTIVE TASK *)
(* ----- *)
(* Hidden system: S5 *)
(* *)
(* Primitive alphabet: *)
(*  $\theta$  = swap positions 1,2 *)

```

```

(* 1 = swap positions 2,3 *)
(* 2 = swap positions 3,4 *)
(* 3 = swap positions 4,5 *)
(*)
(* Query symbols: {0,1,2,3,4} *)
(* Full signature: 5 x 5 = 25 final binary queries *)
(*)
(* LEARNER KNOWS: *)
(* - primitive alphabet *)
(* - query symbols *)
(* - state behavior belongs to the bijection hypothesis class *)
(* - one oracle query returns one final binary Y *)
(*)
(* LEARNER IS NOT GIVEN: *)
(* - hidden state *)
(* - hidden permutation *)
(* - hidden transition table *)
(* - hidden reachable-state count *)
(* - intermediate execution trace *)
(* - the number 120 as a discovery stopping condition *)
(*)
(* TRUE S5 IS USED ONLY AFTER FREEZE. *)
(* ===== *)

ClearAll["Global`S114A*"];
ClearAll["Global`s114A*"];

s114AResult =
Catch[

(

(* ===== *)
(* 0. FAIL SAFE *)
(* ===== *)

S114AFail[msg_] := (
  Print["STOP: ", msg];
  Throw[$Failed, "S114A_STOP"];
);

Print["====="];
Print["S114A FROZEN-POLICY PROSPECTIVE S5 DISCOVERY"];
Print["====="];

(* ===== *)
(* 1. TASK INTERFACE *)
(* ===== *)

alphabetS114A =
  {0, 1, 2, 3};

```



```

querySymbolsS114A =
    {0, 1, 2, 3, 4};

allQueryPairsS114A =
    Tuples[
        querySymbolsS114A,
        2
    ];

queryIndexS114A =
    AssociationThread[
        querySymbolsS114A,
        Range[5]
    ];

Print[
    "Primitive alphabet = ",
    alphabetS114A
];

Print[
    "Query symbols = ",
    querySymbolsS114A
];

Print[
    "Full observable queries per state = ",
    Length[allQueryPairsS114A]
];

Print["S113B QUERY POLICY IS FROZEN."];
Print["NO FULL-25-QUERY SCAN DURING DISCOVERY."];
Print["HIDDEN REACHABLE-STATE COUNT IS NOT A STOPPING CONDITION."];
Print["LEARNER RECEIVES FINAL BINARY Y ONLY."];
Print["TRUE S5 IS POST-HOC ONLY."];

(* ===== *)
(* 2. PROSPECTIVE HOLDOUT INPUTS *)
(* *)
(* Created BEFORE discovery. *)
(* No environment outputs are queried yet. *)
(* ===== *)

holdoutSeedS114A =
    1140501;

holdoutDepthsS114A =

```

```

{
  16,
  32,
  64,
  128
};

holdoutSequencesPerDepthS114A =
  100;

prospectiveHoldoutSeqsS114A =
  BlockRandom[

    SeedRandom[
      holdoutSeedS114A
    ];

    Flatten[
      Table[

        Table[

          <|

            "Depth" ->
              depth,

            "Sequence" ->
              RandomChoice[
                alphabetS114A,
                depth
              ]

          |>,

          {
            holdoutSequencesPerDepthS114A
          }
        ],

        {
          depth,
          holdoutDepthsS114A
        }
      ],

      1
    ]
  ];

Print[
  "Prospective holdout sequences created = ",

```

```

Length[prospectiveHoldoutSeqsS114A]
];

Print[
  "Holdout depths = ",
  holdoutDepthsS114A
];

Print["HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED."];

(* ===== *)
(* 3. HIDDEN ENVIRONMENT *)
(* *)
(* Environment-only implementation. *)
(* Learner may access ONLY S114AOracleY. *)
(* ===== *)

S114AEnvOp[0] :=
  {2, 1, 3, 4, 5};

S114AEnvOp[1] :=
  {1, 3, 2, 4, 5};

S114AEnvOp[2] :=
  {1, 2, 4, 3, 5};

S114AEnvOp[3] :=
  {1, 2, 3, 5, 4};

S114AEnvStep[
  hiddenState_List,
  token_Integer
] :=
  S114AEnvOp[token][[
    hiddenState
  ]];

S114AEnvFinalState[
  seq_List
] :=
  Fold[
    S114AEnvStep,
    {1, 2, 3, 4, 5},
    seq
  ];

S114AEnvRawY[

```

```

seq_List,
start_,
target_
] :=
Module[
{
  hiddenState,
  startIndex,
  targetIndex
},

hiddenState =
  S114AEnvFinalState[
    seq
  ];

startIndex =
  Lookup[
    queryIndexS114A,
    start,
    Missing["UnknownStart"]
  ];

targetIndex =
  Lookup[
    queryIndexS114A,
    target,
    Missing["UnknownTarget"]
  ];

If[
  MissingQ[startIndex] ||
  MissingQ[targetIndex],

  Return[
    Missing["UnknownQuery"]
  ]
];

Boole[
  hiddenState[[startIndex]] ===
  targetIndex
];

(* ===== *)
(* 4. MEMBERSHIP QUERY ORACLE *)
(* ===== *)

oracleCacheS114A =

```

```

<||>;

oracleUniqueQueryCountS114A =
  0;

oracleTotalCallCountS114A =
  0;

S114AOracleKey[
  seq_list,
  start_,
  target_
] :=
  ToString[
    InputForm[
      {
        seq,
        start,
        target
      }
    ]
  ];

S114AOracleY[
  seq_list,
  start_,
  target_
] :=
  Module[
    {
      key,
      y
    },
    oracleTotalCallCountS114A++;

    key =
      S114AOracleKey[
        seq,
        start,
        target
      ];

    If[
      KeyExistsQ[
        oracleCacheS114A,
        key
      ],

```

```

Return[
  Lookup[
    oracleCacheS114A,
    key
  ]
];

y =
  S114AEnvRawY[
    seq,
    start,
    target
  ];

If[
  !MemberQ[
    {0, 1},
    y
  ],
  S114AFail[
    "Environment returned a non-binary output."
  ]
];

oracleUniqueQueryCountS114A++;

AssociateTo[
  oracleCacheS114A,
  key -> y
];

y
];

(* ===== *)
(* 5. FROZEN BEHAVIOR-HYPOTHESIS CLASS *)
(* *)
(* Same structural assumption as S113B, generalized from 4 to *)
(* 5 query symbols. *)
(* *)
(* IMPORTANT: *)
(* this is the possible behavior class, NOT a statement that *)
(* every candidate must be reachable. *)
(* *)
(* The number of reachable states is NOT used to stop search. *)
(* ===== *)

behaviorHypothesesS114A =

```

```

Permutations[
  querySymbolsS114A
];

behaviorHypothesisCountS114A =
  Length[
    behaviorHypothesesS114A
  ];

Print[
  "Task-level behavioral hypotheses = ",
  behaviorHypothesisCountS114A
];

S114ACandidateY[
  candidate_List,
  start_,
  target_
] :=
Module[
  {
    startIndex
  },
  startIndex =
    Lookup[
      queryIndexS114A,
      start,
      Missing["UnknownStart"]
    ];

  If[
    MissingQ[startIndex],
    Return[
      Missing["UnknownStart"]
    ]
  ];

  Boole[
    candidate[[startIndex]] ==
      target
  ]
];

S114ASignatureFromBijection[
  candidate_List
] :=
Table[

```

```

S114ACandidateY[
  candidate,
  pair[[1]],
  pair[[2]]
],

{
  pair,
  allQueryPairsS114A
}
];

S114ABehaviorKey[
  candidate_List
] :=
ToString[
  InputForm[
    candidate
  ]
];

(* ===== *)
(* 6. FROZEN S113B ADAPTIVE QUERY POLICY *)
(* *)
(* EXACT SAME RULE: *)
(* *)
(*  $q^* = \operatorname{argmin}_q \max(N0(q), N1(q))$  *)
(* *)
(* Tie-break: imbalance, then deterministic query order. *)
(* ===== *)

S114AChooseQuery[
  candidates_List,
  askedQueries_List
] :=
Module[
  {
    available,
    scored,
    informative,
    best
  },

  available =
    Select[
      allQueryPairsS114A,

      !MemberQ[
        askedQueries,
        #
      ] &
    ];

```



```

If[
  Length[available] == 0,

  Return[
    Missing[
      "NoAvailableQuery"
    ]
  ]
];

scored =
  Table[

    Module[
      {
        answers,
        n0,
        n1
      },

      answers =
        Table[

          S114ACandidateY[
            candidate,
            query[[1]],
            query[[2]]
          ],

          {
            candidate,
            candidates
          }
        ];

      n0 =
        Count[
          answers,
          0
        ];

      n1 =
        Count[
          answers,
          1
        ];

      <|

```

```

    "Query" ->
      query,

    "N0" ->
      n0,

    "N1" ->
      n1,

    "WorstRemaining" ->
      Max[
        n0,
        n1
      ],

    "Imbalance" ->
      Abs[
        n0 - n1
      ]

    |>

  ],

  {
    query,
    available
  }
];

informative =
  Select[
    scored,

    And[
      #["N0"] > 0,
      #["N1"] > 0
    ] &
  ];

If[
  Length[informative] == 0,

  Return[
    Missing[
      "NoInformativeQuery"
    ]
  ]
];

best =
  First[
    SortBy[

```

```

informative,

Function[
  row,

  {
    row["WorstRemaining"],
    row["Imbalance"],
    ToString[
      InputForm[
        row["Query"]
      ]
    ]
  }
]
];

best["Query"]
];

(* ===== *)
(* 7. IDENTIFY A PREFIX BY ADAPTIVE FINAL-OUTPUT QUERIES *)
(* ===== *)

identificationLogS114A =
  {};

identifiedSequenceKeysS114A =
  <||>;

S114AIdentifyBehavior[
  seq_List
] :=
Module[
  {
    candidates,
    asked,
    answers,
    query,
    y,
    beforeCount,
    afterCount,
    result,
    uniqueBefore,
    uniqueAfter,
    seqKey
  },

  candidates =
    behaviorHypothesesS114A;

```

```

asked =
  {};

answers =
  {};

uniqueBefore =
  oracleUniqueQueryCountS114A;

While[
  Length[candidates] > 1,

  query =
    S114AChooseQuery[
      candidates,
      asked
    ];

  If[
    MissingQ[query],

    S114AFail[
      StringJoin[
        "Unable to distinguish remaining hypotheses for sequence ",
        ToString[
          InputForm[
            seq
          ]
        ]
      ]
    ]
  ];

  beforeCount =
    Length[
      candidates
    ];

  y =
    S114AOracleY[
      seq,
      query[[1]],
      query[[2]]
    ];

  AppendTo[

```

```

    asked,
    query
];

AppendTo[
  answers,
  y
];

candidates =
  Select[
    candidates,

    S114ACandidateY[
      #,
      query[[1]],
      query[[2]]
    ] ==
    y &
  ];

afterCount =
  Length[
    candidates
  ];

If[
  afterCount == 0,

  S114AFail[
    StringJoin[
      "Observed behavior violates the frozen bijection hypothesis class for seq
      ToString[
        InputForm[
          seq
        ]
      ]
    ]
  ]
];

If[
  afterCount >= beforeCount,

  S114AFail[
    "Adaptive query failed to reduce the candidate set."
  ]
];
];

```

```
result =
  First[
    candidates
  ];

uniqueAfter =
  oracleUniqueQueryCountS114A;

seqKey =
  ToString[
    InputForm[
      seq
    ]
  ];

AssociateTo[
  identifiedSequenceKeysS114A,
  seqKey -> True
];

AppendTo[
  identificationLogS114A,

  <|

    "Sequence" ->
      seq,

    "Depth" ->
      Length[seq],

    "QueriesAsked" ->
      Length[asked],

    "UniqueQueriesAdded" ->
      (
        uniqueAfter -
        uniqueBefore
      ),

    "AskedQueries" ->
      asked,

    "Answers" ->
      answers,

    "IdentifiedBijection" ->
      result

  |>
];
```

```

    result
];

(* ===== *)
(* 8. INITIAL STATE *)
(* ===== *)

behaviorToStateS114A =
  <||>;

stateBijectionS114A =
  <||>;

stateSignatureS114A =
  <||>;

stateRepresentativeSeqS114A =
  <||>;

transitionByStateS114A =
  <||>;

discoveryRowsS114A =
  {};

S114ATransitionKey[
  state_Integer,
  token_Integer
] :=
  ToString[
    InputForm[
      {
        state,
        token
      }
    ]
  ];

rootBehaviorS114A =
  S114AIdentifyBehavior[
    {}
  ];

rootBehaviorKeyS114A =
  S114ABehaviorKey[

```

```

    rootBehaviorS114A
];

stateCountS114A =
    1;

startStateS114A =
    1;

AssociateTo[
    behaviorToStateS114A,
    rootBehaviorKeyS114A -> 1
];

AssociateTo[
    stateBijectionS114A,
    1 -> rootBehaviorS114A
];

AssociateTo[
    stateSignatureS114A,
    1 ->
        S114ASignatureFromBijection[
            rootBehaviorS114A
        ]
];

AssociateTo[
    stateRepresentativeSeqS114A,
    1 -> {}
];

stateQueueS114A =
    {1};

(* ===== *)
(* 9. FROZEN-POLICY TRANSITION-CLOSURE DISCOVERY *)
(* *)
(* NO hidden-state-count stopping rule. *)
(* Search terminates only when no new behavior state remains. *)
(* ===== *)

Print["====="];
Print["FROZEN-POLICY ACTIVE S5 DISCOVERY"];
Print["====="];

discoveryTimingS114A =

```



```
AbsoluteTiming[

While[
  Length[
    stateQueueS114A
  ] > 0,

  currentStateS114A =
    First[
      stateQueueS114A
    ];

  stateQueueS114A =
    Rest[
      stateQueueS114A
    ];

  currentRepresentativeS114A =
    Lookup[
      stateRepresentativeSeqS114A,
      currentStateS114A
    ];

  Do[

    successorSeqS114A =
      Append[
        currentRepresentativeS114A,
        token
      ];

    successorBehaviorS114A =
      S114AIdentifyBehavior[
        successorSeqS114A
      ];

    successorBehaviorKeyS114A =
      S114ABehaviorKey[
        successorBehaviorS114A
      ];

    newStateQ114A =
      !KeyExistsQ[
        behaviorToStateS114A,
        successorBehaviorKeyS114A
      ];

    If[
```

```
newStateQ114A,  
  
stateCountS114A++;  
  
successorStateS114A =  
    stateCountS114A;  
  
AssociateTo[  
    behaviorToStateS114A,  
    successorBehaviorKeyS114A ->  
        successorStateS114A  
];  
  
AssociateTo[  
    stateBijectionS114A,  
    successorStateS114A ->  
        successorBehaviorS114A  
];  
  
AssociateTo[  
    stateSignatureS114A,  
    successorStateS114A ->  
        S114ASignatureFromBijection[  
            successorBehaviorS114A  
        ]  
];  
  
AssociateTo[  
    stateRepresentativeSeqS114A,  
    successorStateS114A ->  
        successorSeqS114A  
];  
  
AppendTo[  
    stateQueueS114A,  
    successorStateS114A  
],  
  
successorStateS114A =  
    Lookup[  
        behaviorToStateS114A,  
        successorBehaviorKeyS114A  
    ]  
];  
  
AssociateTo[  
    transitionByStateS114A,
```

```

    S114ATransitionKey[
      currentStateS114A,
      token
    ] ->
      successorStateS114A
  ];

  AppendTo[
    discoveryRowsS114A,

    <|

      "SourceState" ->
        currentStateS114A,

      "Token" ->
        token,

      "DestinationState" ->
        successorStateS114A,

      "SuccessorSeq" ->
        successorSeqS114A,

      "CreatedNewState" ->
        newStateQ114A

    |>
  ];

  ,
  {
    token,
    alphabetS114A
  }
  ];

  (* purely defensive safety guard *)

  If[
    stateCountS114A > 10000,

    S114AFail[
      "State-space safety guard exceeded 10000 states."
    ]
  ];

];

];

Print[

```

```

"Discovery seconds = ",
N[
    discoveryTimingS114A[[1]]
]
];

Print[
    "Observable states discovered = ",
    stateCountS114A
];

Print[
    "Transitions discovered = ",
    Length[
        transitionByStateS114A
    ]
];

(* ===== *)
(* 10. QUERY-EFFICIENCY STATISTICS *)
(* ===== *)

uniqueIdentifiedSequencesS114A =
    Length[
        identifiedSequenceKeysS114A
    ];

fullSignatureBaselineQueriesS114A =
    uniqueIdentifiedSequencesS114A *
    Length[
        allQueryPairsS114A
    ];

membershipQueriesBeforeFreezeS114A =
    oracleUniqueQueryCountS114A;

querySavingsS114A =
    fullSignatureBaselineQueriesS114A -
    membershipQueriesBeforeFreezeS114A;

querySavingsFractionS114A =
    N[
        querySavingsS114A /
        fullSignatureBaselineQueriesS114A
    ];

queriesPerIdentificationS114A =
    (

```

```
#["QueriesAsked"] &
) /@
identificationLogS114A;

meanQueriesPerIdentificationS114A =
  N[
    Mean[
      queriesPerIdentificationS114A
    ]
  ];

minQueriesPerIdentificationS114A =
  Min[
    queriesPerIdentificationS114A
  ];

maxQueriesPerIdentificationS114A =
  Max[
    queriesPerIdentificationS114A
  ];

queryCountHistogramS114A =
  Counts[
    queriesPerIdentificationS114A
  ];

informationLowerBoundS114A =
  N[
    Log[
      2,
      behaviorHypothesisCountS114A
    ]
  ];

worstCaseBinaryLowerBoundS114A =
  Ceiling[
    informationLowerBoundS114A
  ];

meanQueryToInfoRatioS114A =
  N[
    meanQueriesPerIdentificationS114A /
    informationLowerBoundS114A
  ];

Print[
  "Unique identified sequences = ",
  uniqueIdentifiedSequencesS114A
```

```
];

Print[
  "Full-25-query baseline = ",
  fullSignatureBaselineQueriesS114A
];

Print[
  "Actual membership queries = ",
  membershipQueriesBeforeFreezeS114A
];

Print[
  "Queries saved = ",
  querySavingsS114A
];

Print[
  "Query savings fraction = ",
  querySavingsFractionS114A
];

Print[
  "Mean queries / identification = ",
  meanQueriesPerIdentificationS114A
];

Print[
  "Min / Max queries = ",
  minQueriesPerIdentificationS114A,
  " / ",
  maxQueriesPerIdentificationS114A
];

Print[
  "Query-count histogram = ",
  queryCountHistogramS114A
];

Print[
  "Information lower bound log2(120) = ",
  informationLowerBoundS114A
];

Print[
  "Binary worst-case lower bound = ",
  worstCaseBinaryLowerBoundS114A
];
```

```

];

Print[
  "Mean-query / information-bound ratio = ",
  meanQueryToInfoRatioS114A
];

(* ===== *)
(* 11. STRUCTURAL AUDITS BEFORE FREEZE *)
(* ===== *)

stateBehaviorAuditS114A =
  AllTrue[
    Values[
      stateBijectionS114A
    ],
    Function[
      bij,
      And[
        ListQ[bij],
        Length[bij] == 5,
        Sort[bij] ==
          querySymbolsS114A
      ]
    ]
  ];

distinctBehaviorCountS114A =
  Length[
    DeleteDuplicates[
      Values[
        stateBijectionS114A
      ]
    ]
  ];

distinctSignatureCountS114A =
  Length[
    DeleteDuplicates[
      Values[
        stateSignatureS114A
      ]
    ]
  ];

expectedTransitionCountS114A =
  stateCountS114A *
  Length[
    alphabetS114A
  ]

```

```

];

transitionCompletenessS114A =
  N[
    Length[
      transitionByStateS114A
    ] /
    expectedTransitionCountS114A
  ];

Print[
  "State behavior audit = ",
  stateBehaviorAuditS114A
];

Print[
  "Distinct behavioral states = ",
  distinctBehaviorCountS114A
];

Print[
  "Distinct derived signatures = ",
  distinctSignatureCountS114A
];

Print[
  "Transitions = ",
  Length[
    transitionByStateS114A
  ],
  " / ",
  expectedTransitionCountS114A
];

Print[
  "Transition completeness = ",
  transitionCompletenessS114A
];

If[
  !TrueQ[
    stateBehaviorAuditS114A
  ],
  S114AFail[
    "At least one discovered behavior is not a valid 5-symbol bijection."
  ]
];

```



```

If[
  Abs[
    transitionCompletenessS114A - 1.
  ] > 10^-12,

  S114AFail[
    "Transition table is incomplete."
  ]
];

(* ===== *)
(* 12. LEARNED MACHINE EXECUTION *)
(* ===== *)

S114AExecuteState[
  seq_List
] :=
Module[
  {
    state
  },

  state =
    startStateS114A;

  Do[

    state =
      Lookup[
        transitionByStateS114A,

        S114ATransitionKey[
          state,
          token
        ],

        Missing[
          "UnknownTransition"
        ]
      ];

    If[
      MissingQ[state],
      Return[state]
    ];

  ,
  {
    token,
    seq
  }
]

```

```

];

state
];

S114ALearnedSignature[
  seq_List
] :=
Module[
  {
    state
  },

  state =
    S114AExecuteState[
      seq
    ];

  If[
    MissingQ[state],

    Return[
      Missing[
        "UnknownTransition"
      ]
    ]
  ];

  Lookup[
    stateSignatureS114A,
    state,
    Missing[
      "UnknownState"
    ]
  ]
];

S114APredict[
  seq_List,
  start_,
  target_
] :=
Module[
  {
    state,
    behavior,
    startIndex
  },

```

```
state =
  S114AExecuteState[
    seq
  ];

If[
  MissingQ[state],

  Return[
    Missing[
      "UnknownTransition"
    ]
  ]
];

behavior =
  Lookup[
    stateBijectionS114A,
    state,
    Missing[
      "UnknownState"
    ]
  ];

startIndex =
  Lookup[
    queryIndexS114A,
    start,
    Missing[
      "UnknownStart"
    ]
  ];

If[
  MissingQ[behavior] ||
  MissingQ[startIndex],

  Return[
    Missing[
      "Unavailable"
    ]
  ]
];

Boole[
  behavior[[startIndex]] ==
  target
];
```

```

(* ===== *)
(* 13. PROSPECTIVE HOLDOUT LEAKAGE AUDIT *)
(* *)
(* Cache inspection only. No oracle call. *)
(* ===== *)

S114AHoldoutTouchedQ[
  seq_List
] :=
  AnyTrue[
    allQueryPairsS114A,

    Function[
      pair,

      KeyExistsQ[
        oracleCacheS114A,

        S114AOracleKey[
          seq,
          pair[[1]],
          pair[[2]]
        ]
      ]
    ]
  ];

holdoutTouchedFlagsS114A =
(
  S114AHoldoutTouchedQ[
    #["Sequence"]
  ] &
) /@
prospectiveHoldoutSeqsS114A;

holdoutTouchedBeforeFreezeS114A =
Count[
  holdoutTouchedFlagsS114A,
  True
];

Print[
  "Holdout sequences touched before freeze = ",
  holdoutTouchedBeforeFreezeS114A,
  " / ",
  Length[
    prospectiveHoldoutSeqsS114A
  ]
];

(* ===== *)
(* 14. FREEZE *)

```

```

(* ===== *)

machineHashS114A =
  Hash[
    {
      stateBijectionS114A,
      stateSignatureS114A,
      stateRepresentativeSeqS114A,
      transitionByStateS114A,
      startStateS114A
    },

    "SHA256",
    "HexString"
  ];

Print["====="];
Print["S114A S5 MACHINE FROZEN"];

Print[
  "States = ",
  stateCountS114A
];

Print[
  "Transitions = ",
  Length[
    transitionByStateS114A
  ]
];

Print[
  "Membership queries = ",
  membershipQueriesBeforeFreezeS114A
];

Print[
  "Full-signature baseline = ",
  fullSignatureBaselineQueriesS114A
];

Print[
  "Query savings = ",
  100. querySavingsFractionS114A,
  "%"
];

Print[
  "Machine hash = ",

```

```

machineHashS114A
];

Print["NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT."];
Print["====="];

(* ===== *)
(* 15. OPEN PROSPECTIVE LONG-LENGTH HOLDOUT *)
(* ===== *)

Print["====="];
Print["OPENING PROSPECTIVE S5 HOLDOUT AFTER FREEZE"];
Print["====="];

holdoutRowsS114A =
  Table[
    Module[
      {
        seq,
        learnedSignature,
        oracleSignature
      },
      seq =
        row[
          "Sequence"
        ];

      learnedSignature =
        S114ALearnedSignature[
          seq
        ];

      oracleSignature =
        Table[
          S114AOracleY[
            seq,
            pair[[1]],
            pair[[2]]
          ],
          {
            pair,
            allQueryPairsS114A
          }
        ];
    ];
  ];

```

```

<|
  "Depth" ->
    row["Depth"],

  "LearnedState" ->
    S114AExecuteState[
      seq
    ],

  "SignatureMatch" ->
    (
      learnedSignature ===
      oracleSignature
    )

|>

],

{
  row,
  prospectiveHoldoutSeqsS114A
}
];

holdoutMatchesS114A =
  Count[
    (
      TrueQ[
        #["SignatureMatch"]
      ] &
    ) /@
    holdoutRowsS114A,
    True
  ];

holdoutAccuracyS114A =
  N[
    holdoutMatchesS114A /
    Length[
      holdoutRowsS114A
    ]
  ];

holdoutPerDepthS114A =
  Table[

    Module[
      {
        rows,
        matches
      },

```

```

rows =
  Select[
    holdoutRowsS114A,
    #["Depth"] === depth &
  ];

matches =
  Count[
    (
      TrueQ[
        #["SignatureMatch"]
      ] &
    ) /@
    rows,
    True
  ];

<|

  "Depth" ->
    depth,

  "Sequences" ->
    Length[
      rows
    ],

  "Matches" ->
    matches,

  "Accuracy" ->
    N[
      matches /
      Length[
        rows
      ]
    ]

|>

],

{
  depth,
  holdoutDepthsS114A
}
];

Print[
  "Prospective holdout = ",
  holdoutMatchesS114A,

```



```

" / ",
Length[
  holdoutRowsS114A
]
];

Print[
  "Prospective holdout accuracy = ",
  holdoutAccuracyS114A
];

(* ===== *)
(* 16. POST-FREEZE EXHAUSTIVE SHORT AUDIT *)
(* *)
(* Alphabet size 4. *)
(* Lengths 0..5: *)
(* 1 + 4 + 16 + 64 + 256 + 1024 = 1365 sequences. *)
(* ===== *)

auditMaxLengthS114A =
  5;

auditSequencesS114A =
  Flatten[
    Table[
      Tuples[
        alphabetS114A,
        len
      ],
      {
        len,
        0,
        auditMaxLengthS114A
      }
    ],
    1
  ];

auditRowsS114A =
  Table[
    Module[
      {
        learnedSignature,
        oracleSignature
      },

      learnedSignature =

```

```

S114ALearnedSignature[
  seq
];

oracleSignature =
  Table[

    S114AOracleY[
      seq,
      pair[[1]],
      pair[[2]]
    ],

    {
      pair,
      allQueryPairsS114A
    }
  ];

<|

  "Length" ->
    Length[
      seq
    ],

  "LearnedState" ->
    S114AExecuteState[
      seq
    ],

  "SignatureMatch" ->
    (
      learnedSignature ==
      oracleSignature
    )

|>

],

{
  seq,
  auditSequencesS114A
}
];

auditMatchesS114A =
  Count[
    (
      TrueQ[
        #["SignatureMatch"]
      ] &

```

```

    ) /@
    auditRowsS114A,
    True
];

auditAccuracyS114A =
  N[
    auditMatchesS114A /
    Length[
      auditRowsS114A
    ]
  ];

auditPerLengthS114A =
  Table[

    Module[
      {
        rows,
        matches
      },

      rows =
        Select[
          auditRowsS114A,
          #["Length"] === len &
        ];

      matches =
        Count[
          (
            TrueQ[
              #["SignatureMatch"]
            ] &
          ) /@
            rows,
          True
        ];

      <|

        "Length" ->
          len,

        "Sequences" ->
          Length[
            rows
          ],

        "Matches" ->
          matches,

```

```

        "Accuracy" ->
            N[
                matches /
                Length[
                    rows
                ]
            ]

        |>

    ],

    {
        len,
        0,
        auditMaxLengthS114A
    }
];

Print[
    "Exhaustive length 0..",
    auditMaxLengthS114A,
    " = ",
    auditMatchesS114A,
    " / ",
    Length[
        auditRowsS114A
    ]
];

(* ===== *)
(* 17. VERIFY FROZEN HASH *)
(* ===== *)

recomputedHashS114A =
    Hash[
        {
            stateBijectionS114A,
            stateSignatureS114A,
            stateRepresentativeSeqS114A,
            transitionByStateS114A,
            startStateS114A
        },

        "SHA256",
        "HexString"
    ];

frozenHashMatchesS114A =
    recomputedHashS114A ===
    machineHashS114A;

```

```

Print[
  "Frozen hash unchanged = ",
  frozenHashMatchesS114A
];

If[
  !TrueQ[
    frozenHashMatchesS114A
  ],

  S114AFail[
    "Frozen machine changed after freeze."
  ]
];

(* ===== *)
(* 18. POST-HOC TRUE S5 AUDIT BEGINS *)
(* *)
(* Nothing below is allowed to alter the Learned machine. *)
(* ===== *)

Print["====="];
Print["POST-HOC TRUE S5 ISOMORPHISM AUDIT"];
Print["====="];

trueStatesS114A =
  Permutations[
    {1, 2, 3, 4, 5}
  ];

trueStateCountS114A =
  Length[
    trueStatesS114A
  ];

trueStartStateS114A =
  {1, 2, 3, 4, 5};

Print[
  "Post-hoc true S5 states = ",
  trueStateCountS114A
];

S114ATrueQuery[
  hiddenState_List,
  start_,
  target_
] :=

```

```

Module[
  {
    startIndex,
    targetIndex
  },

  startIndex =
    Lookup[
      queryIndexS114A,
      start,
      Missing["UnknownStart"]
    ];

  targetIndex =
    Lookup[
      queryIndexS114A,
      target,
      Missing["UnknownTarget"]
    ];

  If[
    MissingQ[startIndex] ||
    MissingQ[targetIndex],

    Return[
      Missing[
        "UnknownQuery"
      ]
    ]
  ];

  Boole[
    hiddenState[[startIndex]] ===
    targetIndex
  ]
];

S114ATrueSignature[
  hiddenState_List
] :=
  Table[

    S114ATrueQuery[
      hiddenState,
      pair[[1]],
      pair[[2]]
    ],

    {
      pair,
      allQueryPairsS114A
    }
  ]

```

```

    }
];

trueDistinctSignatureCountS114A =
  Length[
    DeleteDuplicates[
      S114ATrueSignature /@
      trueStatesS114A
    ]
  ];

Print[
  "Distinct true S5 behavior signatures = ",
  trueDistinctSignatureCountS114A
];

(* ===== *)
(* 19. LEARNED STATE -> TRUE S5 STATE *)
(* ===== *)

stateCorrespondenceS114A =
  Table[
    Module[
      {
        learnedSignature,
        matches
      },
      learnedSignature =
        Lookup[
          stateSignatureS114A,
          learnedState
        ];

      matches =
        Select[
          trueStatesS114A,

          S114ATrueSignature[#] ===
            learnedSignature &
        ];

      <|

        "LearnedState" ->
          learnedState,

        "RepresentativeSeq" ->
          Lookup[

```

```

        stateRepresentativeSeqS114A,
        learnedState
    ],

    "RepresentativeLength" ->
        Length[
            Lookup[
                stateRepresentativeSeqS114A,
                learnedState
            ]
        ],

    "LearnedBijection" ->
        Lookup[
            stateBijectionS114A,
            learnedState
        ],

    "TrueMatches" ->
        matches,

    "MatchCount" ->
        Length[
            matches
        ],

    "Functional" ->
        (
            Length[
                matches
            ] == 1
        )

    |>

],

{
    learnedState,
    Range[
        stateCountS114A
    ]
}
];

learnedToTrueFunctionalS114A =
    AllTrue[
        stateCorrespondenceS114A,

        Function[
            row,
            TrueQ[
                row["Functional"]
            ]
        ]
    ]

```



```

];

assignedTrueStatesS114A =
  If[
    learnedToTrueFunctionalS114A,

    (
      First[
        #["TrueMatches"]
      ] &
    ) /@
      stateCorrespondenceS114A,

    {}
  ];

trueToLearnedFunctionalS114A =
  If[
    learnedToTrueFunctionalS114A,

    AllTrue[
      trueStatesS114A,

      Function[
        trueState,

        Count[
          assignedTrueStatesS114A,
          trueState
        ] == 1
      ]
    ],

    False
  ];

stateBijectionPassedS114A =
  And[
    learnedToTrueFunctionalS114A,
    trueToLearnedFunctionalS114A,
    stateCountS114A ==
      trueStateCountS114A
  ];

Print[
  "Learned -> true functional = ",
  learnedToTrueFunctionalS114A
];

Print[
  "True -> learned functional = ",

```

```

trueToLearnedFunctionalS114A
];

Print[
  "State-space bijection = ",
  stateBijectionPassedS114A
];

(* ===== *)
(* 20. BUILD POST-HOC PHI *)
(* ===== *)

phiS114A =
  If[
    stateBijectionPassedS114A,

    Association[
      Table[

        row["LearnedState"] ->
          First[
            row["TrueMatches"]
          ],

        {
          row,
          stateCorrespondenceS114A
        }
      ]
    ],

    <||>
  ];

startStateMatchS114A =
  If[
    stateBijectionPassedS114A,

    Lookup[
      phiS114A,
      startStateS114A,
      Missing["NoMapping"]
    ] ==
      trueStartStateS114A,

    False
  ];

Print[
  "Start-state correspondence = ",
  startStateMatchS114A
];

```

```

(* ===== *)
(* 21. EXACT TRANSITION ISOMORPHISM *)
(* *)
(* Post-hoc expected if exact: *)
(* 120 states x 4 primitive tokens = 480 checks. *)
(* ===== *)

S114ALearnedStep[
  learnedState_Integer,
  token_Integer
] :=
  Lookup[
    transitionByStateS114A,

    S114ATransitionKey[
      learnedState,
      token
    ],

    Missing[
      "UnknownTransition"
    ]
  ];

transitionAuditRowsS114A =
  Flatten[
    Table[

      Module[
        {
          learnedDestination,
          trueSource,
          mappedLearnedDestination,
          trueDestination
        },

        learnedDestination =
          S114ALearnedStep[
            learnedState,
            token
          ];

        trueSource =
          Lookup[
            phiS114A,
            learnedState,
            Missing[
              "NoPhi"
            ]
          ]
      ];
    ],
    1
  ];

```

```
mappedLearnedDestination =  
  If[  
    IntegerQ[  
      learnedDestination  
    ],  
  
    Lookup[  
      phiS114A,  
      learnedDestination,  
      Missing[  
        "NoDestPhi"  
      ]  
    ],  
  
    Missing[  
      "NoLearnedDestination"  
    ]  
  ];  
  
trueDestination =  
  If[  
    ListQ[  
      trueSource  
    ],  
  
    S114AEnvStep[  
      trueSource,  
      token  
    ],  
  
    Missing[  
      "NoTrueSource"  
    ]  
  ];  
  
<|  
  
"LearnedSource" ->  
  learnedState,  
  
"Token" ->  
  token,  
  
"LearnedDestination" ->  
  learnedDestination,  
  
"TrueSource" ->  
  trueSource,  
  
"MappedLearnedDestination" ->  
  mappedLearnedDestination,  
  
"TrueDestination" ->
```

```

        trueDestination,

        "Match" ->
        (
            mappedLearnedDestination ===
            trueDestination
        )

    |>

],

{
    learnedState,
    Range[
        stateCountS114A
    ]
},

{
    token,
    alphabetS114A
}
],

1
];

```

```

transitionTotalS114A =
    Length[
        transitionAuditRowsS114A
    ];

```

```

transitionMatchesS114A =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        transitionAuditRowsS114A,
        True
    ];

```

```

transitionIsomorphismPassedS114A =
    transitionMatchesS114A ==
    transitionTotalS114A;

```

```

Print[
    "Transition correspondence = ",
    transitionMatchesS114A,
    " / ",

```

```

transitionTotalS114A
];

(* ===== *)
(* 22. COMPLETE STATE x QUERY BEHAVIOR AUDIT *)
(* *)
(* Post-hoc expected if exact: *)
(* 120 x 25 = 3000 checks. *)
(* ===== *)

queryAuditRowsS114A =
  Flatten[
    Table[

      Module[
        {
          trueState,
          learnedBehavior
        },

        trueState =
          Lookup[
            phiS114A,
            learnedState,
            Missing[
              "NoPhi"
            ]
          ]
      ];

      learnedBehavior =
        Lookup[
          stateBijectionS114A,
          learnedState
        ]
      ];

    Table[

      Module[
        {
          learnedY,
          trueY
        },

        learnedY =
          S114ACandidateY[
            learnedBehavior,
            pair[[1]],
            pair[[2]]
          ]
      ];
    ]
  ]

```

```

trueY =
  If[
    ListQ[
      trueState
    ],

    S114ATrueQuery[
      trueState,
      pair[[1]],
      pair[[2]]
    ],

    Missing[
      "NoTrueState"
    ]
  ];

<|

  "LearnedState" ->
    learnedState,

  "Start" ->
    pair[[1]],

  "Target" ->
    pair[[2]],

  "LearnedY" ->
    learnedY,

  "TrueY" ->
    trueY,

  "Match" ->
    (
      learnedY ==
      trueY
    )

|>

],

{
  pair,
  allQueryPairsS114A
}

],

{
  learnedState,
  Range[

```

```

        stateCountS114A
    ]
}
],
1
];

queryTotalS114A =
    Length[
        queryAuditRowsS114A
    ];

queryMatchesS114A =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        queryAuditRowsS114A,
        True
    ];

fullQueryBehaviorPassedS114A =
    queryMatchesS114A ==
    queryTotalS114A;

Print[
    "Complete state/query behavior = ",
    queryMatchesS114A,
    " / ",
    queryTotalS114A
];

(* ===== *)
(* 23. SYNCHRONIZED PRODUCT FIXED-POINT *)
(* ===== *)
(* Explore the entire reachable pair system, not a sampled *)
(* finite input length. *)
(* ===== *)

S114AProductClosure[] :=
Module[
    {
        startPair,
        reached,
        frontier,
        nextPairs,
        newPairs,
        learnedNext,
    }

```



```
    trueNext,  
    guard  
  },  
  
  startPair =  
  {  
    startStateS114A,  
    trueStartStateS114A  
  };  
  
  reached =  
  {  
    startPair  
  };  
  
  frontier =  
  {  
    startPair  
  };  
  
  guard =  
  0;  
  
  While[  
    Length[  
      frontier  
    ] > 0,  
  
    guard++;  
  
    If[  
      guard > 5000,  
  
      Return[$Failed]  
    ];  
  
    nextPairs =  
      DeleteDuplicates[  
        Flatten[  
          Table[  
  
            learnedNext =  
              S114ALearnedStep[  
                pair[[1]],  
                token  
              ];  

```

```

        trueNext =
            S114AEnvStep[
                pair[[2]],
                token
            ];

        {
            learnedNext,
            trueNext
        },

        {
            pair,
            frontier
        },

        {
            token,
            alphabetS114A
        }
    ],
    1
]
];

If[
    AnyTrue[
        nextPairs,

        Function[
            pair,
            !IntegerQ[
                pair[[1]]
            ]
        ]
    ],
    Return[$Failed]
];

newPairs =
    Select[
        nextPairs,

        Function[
            pair,
            !MemberQ[
                reached,
                pair
            ]
        ]
    ]
];

```

```

    reached =
      Join[
        reached,
        newPairs
      ];

    frontier =
      newPairs;

  ];

  reached
];

productPairsS114A =
  S114AProductClosure[];

If[
  productPairsS114A === $Failed,

  S114AFail[
    "Synchronized product exploration failed."
  ]
];

productPairCountS114A =
  Length[
    productPairsS114A
  ];

productLearnedFunctionalS114A =
  AllTrue[
    Range[
      stateCountS114A
    ],
    Function[
      learnedState,

      Length[
        DeleteDuplicates[
          (
            #[[2]] &
          ) /@
          Select[
            productPairsS114A,
            #[[1]] ===
              learnedState &

```

```

    ]
  ]
] == 1
];

productTrueFunctionalS114A =
  AllTrue[
    trueStatesS114A,

    Function[
      trueState,

      Length[
        DeleteDuplicates[
          (
            #[[1]] &
          ) /@
          Select[
            productPairsS114A,
            #[[2]] ===
              trueState &
          ]
        ]
      ] == 1
    ]
  ];

productBijectionPassedS114A =
  And[
    productPairCountS114A ==
      stateCountS114A,

    productLearnedFunctionalS114A,

    productTrueFunctionalS114A
  ];

Print[
  "Synchronized product reachable pairs = ",
  productPairCountS114A
];

Print[
  "Product learned->true functional = ",
  productLearnedFunctionalS114A
];

Print[
  "Product true->learned functional = ",
  productTrueFunctionalS114A
];

```

```

];

(* ===== *)
(* 24. DIRECT BEHAVIORAL STATE ENCODING *)
(* *)
(* Learned symbols = {0,1,2,3,4}. *)
(* True S5 state symbols = {1,2,3,4,5}. *)
(* ===== *)

directEncodingRowsS114A =
  Table[

    Module[
      {
        learnedBehavior,
        mappedTrue
      },

      learnedBehavior =
        Lookup[
          stateBijectionS114A,
          learnedState
        ];

      mappedTrue =
        Lookup[
          phiS114A,
          learnedState,
          Missing[
            "NoPhi"
          ]
        ];

      <|

        "LearnedState" ->
          learnedState,

        "LearnedBijection01234" ->
          learnedBehavior,

        "Converted12345" ->
          (
            learnedBehavior + 1
          ),

        "MappedTrueS5" ->
          mappedTrue,

        "Match" ->
          (
            learnedBehavior + 1 ==

```

```

        mappedTrue
    )

    |>

],

{
  learnedState,
  Range[
    stateCountS114A
  ]
}
];

directEncodingMatchesS114A =
  Count[
    (
      TrueQ[
        #["Match"]
      ] &
    ) /@
      directEncodingRowsS114A,
    True
  ];

directEncodingExactS114A =
  directEncodingMatchesS114A ==
    stateCountS114A;

Print[
  "Direct behavioral encoding = ",
  directEncodingMatchesS114A,
  " / ",
  stateCountS114A
];

(* ===== *)
(* 25. GLOBAL EXACTNESS *)
(* ===== *)

exactMachineIsomorphismS114A =
  And[
    frozenHashMatchesS114A,
    stateBijectionPassedS114A,
    startStateMatchS114A,
    transitionIsomorphismPassedS114A,
    fullQueryBehaviorPassedS114A,
    productBijectionPassedS114A
  ];

```

```

allFiniteSequenceEquivalentS114A =
  exactMachineIsomorphismS114A;

(* ===== *)
(* 26. POST-HOC SUCCESS GATES *)
(* *)
(* TRUE state count is used ONLY HERE. *)
(* Discovery has already finished and machine is frozen. *)
(* ===== *)

stateCountCorrectPostHocS114A =
  stateCountS114A ==
    trueStateCountS114A;

queryEfficientS114A =
  membershipQueriesBeforeFreezeS114A <
    fullSignatureBaselineQueriesS114A;

strongQueryReductionS114A =
  querySavingsFractionS114A >=
    0.50;

prospectiveStrongS114A =
  And[
    holdoutTouchedBeforeFreezeS114A == 0,
    stateCountCorrectPostHocS114A,
    distinctBehaviorCountS114A ==
      trueStateCountS114A,
    distinctSignatureCountS114A ==
      trueStateCountS114A,
    Abs[
      transitionCompletenessS114A - 1.
    ] < 10^-12,
    Abs[
      holdoutAccuracyS114A - 1.
    ] < 10^-12,
    Abs[
      auditAccuracyS114A - 1.
    ] < 10^-12,
    exactMachineIsomorphismS114A,
    queryEfficientS114A
  ];

diagnosisS114A =
  Which[

    prospectiveStrongS114A &&
    strongQueryReductionS114A &&
    directEncodingExactS114A,

    "PROSPECTIVE_S5_SUCCESS_FROZEN_QUERY_EFFICIENT_POLICY_RECOVERS_THE_EXACT_120_"
  ]

```

```

prospectiveStrongS114A &&
directEncodingExactS114A,

    "PROSPECTIVE_S5_SUCCESS_EXACT_MACHINE_RECOVERED_BUT_QUERY_REDUCTION_IS_MODEST

!stateCountCorrectPostHocS114A,

    "PROSPECTIVE_S5_DISCOVERY_DOES_NOT_RECOVER_THE_COMPLETE_TRUE_STATE_SPACE",

!exactMachineIsomorphismS114A,

    "PROSPECTIVE_S5_DISCOVERS_A_MACHINE_BUT_IT_IS_NOT_EXACTLY_ISOMORPHIC_TO_TRUE_

!queryEfficientS114A,

    "PROSPECTIVE_S5_MACHINE_IS_EXACT_BUT_QUERY_POLICY_FAILS_TO_BEAT_THE_FULL_SIGN

holdoutAccuracyS114A < 1.,

    "PROSPECTIVE_S5_MACHINE_FAILS_THE_LONG_SEQUENCE_HOLDOUT",

    True,

    "INCONCLUSIVE"
];

(* ===== *)
(* 27. STATE TABLE *)
(* ===== *)

stateTableS114A =
  Table[

    <|

      "State" ->
        state,

      "RepresentativeSeq" ->
        Lookup[
          stateRepresentativeSeqS114A,
          state
        ],

      "RepresentativeLength" ->
        Length[
          Lookup[
            stateRepresentativeSeqS114A,

```



```

        state
    ]
],

"Behavior" ->
    Lookup[
        stateBijectionS114A,
        state
    ],

"On0" ->
    Lookup[
        transitionByStateS114A,
        S114ATransitionKey[
            state,
            0
        ]
    ],

"On1" ->
    Lookup[
        transitionByStateS114A,
        S114ATransitionKey[
            state,
            1
        ]
    ],

"On2" ->
    Lookup[
        transitionByStateS114A,
        S114ATransitionKey[
            state,
            2
        ]
    ],

"On3" ->
    Lookup[
        transitionByStateS114A,
        S114ATransitionKey[
            state,
            3
        ]
    ],

"TrueS5PostHoc" ->
    Lookup[
        phiS114A,
        state,
        Missing[
            "NoMapping"
        ]
    ]
],
|>,

```

```

    {
      state,
      Range[
        stateCountS114A
      ]
    }
  ];

(* ===== *)
(* 28. FINAL REPORT *)
(* ===== *)

finalS114A =
<|

  "Stage" ->
    "S114A_FrozenPolicyProspectiveS5",

  "ProspectiveTask" ->
    True,

  "S113BQueryPolicyFrozen" ->
    True,

  "TaskLevelBijectionPriorUsed" ->
    True,

  "HiddenReachableStateCountUsedForStopping" ->
    False,

  "LearnerReceivesIntermediateTrueState" ->
    False,

  "LearnerReceivesFinalBinaryYOnly" ->
    True,

  "StatesDiscovered" ->
    stateCountS114A,

  "DistinctBehaviors" ->
    distinctBehaviorCountS114A,

  "DistinctSignatures" ->
    distinctSignatureCountS114A,

  "TransitionsDiscovered" ->
    Length[
      transitionByStateS114A
    ],

  "TransitionCompleteness" ->
    transitionCompletenessS114A,

  "BehaviorHypothesisCount" ->

```

```
behaviorHypothesisCountS114A,  
  
"UniqueIdentifiedSequences" ->  
  uniqueIdentifiedSequencesS114A,  
  
"FullSignatureBaselineQueries" ->  
  fullSignatureBaselineQueriesS114A,  
  
"ActualMembershipQueries" ->  
  membershipQueriesBeforeFreezeS114A,  
  
"QueriesSaved" ->  
  querySavingsS114A,  
  
"QuerySavingsFraction" ->  
  querySavingsFractionS114A,  
  
"MeanQueriesPerIdentification" ->  
  meanQueriesPerIdentificationS114A,  
  
"MinQueriesPerIdentification" ->  
  minQueriesPerIdentificationS114A,  
  
"MaxQueriesPerIdentification" ->  
  maxQueriesPerIdentificationS114A,  
  
"QueryCountHistogram" ->  
  queryCountHistogramS114A,  
  
"InformationLowerBound" ->  
  informationLowerBoundS114A,  
  
"WorstCaseBinaryLowerBound" ->  
  worstCaseBinaryLowerBoundS114A,  
  
"MeanQueryToInformationBoundRatio" ->  
  meanQueryToInfoRatioS114A,  
  
"HoldoutTouchedBeforeFreeze" ->  
  holdoutTouchedBeforeFreezeS114A,  
  
"HoldoutSequences" ->  
  Length[  
    holdoutRowsS114A  
  ],  
  
"HoldoutMatches" ->  
  holdoutMatchesS114A,  
  
"HoldoutAccuracy" ->  
  holdoutAccuracyS114A,  
  
"ExhaustiveAuditMaxLength" ->  
  auditMaxLengthS114A,  
  
"ExhaustiveAuditSequences" ->
```

```
Length[
  auditRowsS114A
],

"ExhaustiveAuditMatches" ->
  auditMatchesS114A,

"ExhaustiveAuditAccuracy" ->
  auditAccuracyS114A,

"TrueS5StatesPostHoc" ->
  trueStateCountS114A,

"StateCountCorrectPostHoc" ->
  stateCountCorrectPostHocS114A,

"StateBijectionPassed" ->
  stateBijectionPassedS114A,

"StartStateMatch" ->
  startStateMatchS114A,

"TransitionMatches" ->
  transitionMatchesS114A,

"TransitionTotal" ->
  transitionTotalS114A,

"TransitionIsomorphismPassed" ->
  transitionIsomorphismPassedS114A,

"QueryBehaviorMatches" ->
  queryMatchesS114A,

"QueryBehaviorTotal" ->
  queryTotalS114A,

"FullQueryBehaviorPassed" ->
  fullQueryBehaviorPassedS114A,

"ProductPairs" ->
  productPairCountS114A,

"ProductBijectionPassed" ->
  productBijectionPassedS114A,

"DirectEncodingMatches" ->
  directEncodingMatchesS114A,

"DirectEncodingExact" ->
  directEncodingExactS114A,

"ExactMachineIsomorphism" ->
  exactMachineIsomorphismS114A,

"AllFiniteSequenceEquivalent" ->
```

```
allFiniteSequenceEquivalentS114A,  
  
"QueryEfficient" ->  
  queryEfficientS114A,  
  
"StrongQueryReduction" ->  
  strongQueryReductionS114A,  
  
"ProspectiveStrong" ->  
  prospectiveStrongS114A,  
  
"Diagnosis" ->  
  diagnosisS114A,  
  
"MachineHash" ->  
  machineHashS114A  
  
|>;  
  
Print["====="];  
Print["S114A COMPLETE"];  
  
Print[  
  "States discovered = ",  
  stateCountS114A  
];  
  
Print[  
  "Distinct behaviors = ",  
  distinctBehaviorCountS114A  
];  
  
Print[  
  "Transitions = ",  
  Length[  
    transitionByStateS114A  
  ],  
  " / ",  
  expectedTransitionCountS114A  
];  
  
Print[  
  "Full-signature query baseline = ",  
  fullSignatureBaselineQueriesS114A  
];  
  
Print[  
  "Actual membership queries = ",  
  membershipQueriesBeforeFreezeS114A  
];
```

```
Print[
  "Queries saved = ",
  querySavingsS114A,
  " (",
  100. querySavingsFractionS114A,
  "%)"
];

Print[
  "Mean queries / identification = ",
  meanQueriesPerIdentificationS114A
];

Print[
  "Information lower bound = ",
  informationLowerBoundS114A
];

Print[
  "Prospective holdout = ",
  holdoutMatchesS114A,
  " / ",
  Length[
    holdoutRowsS114A
  ]
];

Print[
  "Exhaustive audit = ",
  auditMatchesS114A,
  " / ",
  Length[
    auditRowsS114A
  ]
];

Print[
  "Post-hoc state bijection = ",
  stateBijectionPassedS114A
];

Print[
  "Transition correspondence = ",
  transitionMatchesS114A,
  " / ",
  transitionTotalS114A
];
```

```
Print[
  "Query behavior correspondence = ",
  queryMatchesS114A,
  " / ",
  queryTotalS114A
];

Print[
  "Synchronized product pairs = ",
  productPairCountS114A
];

Print[
  "Direct behavioral encoding = ",
  directEncodingMatchesS114A,
  " / ",
  stateCountS114A
];

Print[
  "Exact machine isomorphism = ",
  exactMachineIsomorphismS114A
];

Print[
  "All finite-sequence equivalent = ",
  allFiniteSequenceEquivalentS114A
];

Print[
  "Diagnosis = ",
  diagnosisS114A
];

Print["====="];

finalS114A

),

"S114A_STOP"
];
```

```

=====
S114A FROZEN-POLICY PROSPECTIVE S5 DISCOVERY
=====
Primitive alphabet = {0, 1, 2, 3}
Query symbols = {0, 1, 2, 3, 4}
Full observable queries per state = 25
S113B QUERY POLICY IS FROZEN.
NO FULL-25-QUERY SCAN DURING DISCOVERY.
HIDDEN REACHABLE-STATE COUNT IS NOT A STOPPING CONDITION.
LEARNER RECEIVES FINAL BINARY Y ONLY.
TRUE S5 IS POST-HOC ONLY.
Prospective holdout sequences created = 400
Holdout depths = {16, 32, 64, 128}
HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED.
Task-level behavioral hypotheses = 120
=====
FROZEN-POLICY ACTIVE S5 DISCOVERY
=====
Discovery seconds = 42.2608
Observable states discovered = 120
Transitions discovered = 480
Unique identified sequences = 481
Full-25-query baseline = 12025
Actual membership queries = 3708
Queries saved = 8317
Query savings fraction = 0.691642
Mean queries / identification = 7.70894
Min / Max queries = 4 / 10
Query-count histogram = <|4 -> 9, 5 -> 32, 6 -> 64, 7 -> 104, 8 -> 112,

> 9 -> 96, 10 -> 64|>
Information lower bound log2(120) = 6.90689
Binary worst-case lower bound = 7
Mean-query / information-bound ratio = 1.11612
State behavior audit = True
Distinct behavioral states = 120
Distinct derived signatures = 120
Transitions = 480 / 480
Transition completeness = 1.
Holdout sequences touched before freeze = 0 / 400
=====
S114A S5 MACHINE FROZEN
States = 120
Transitions = 480
Membership queries = 3708
Full-signature baseline = 12025
Query savings = 69.1642%
Machine hash = 1bf23a115d12e19df4799c720ca8a5f4b237c2e2a51d6a5c867cbfd27f3308\

> c1
NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT.
=====
=====
OPENING PROSPECTIVE S5 HOLDOUT AFTER FREEZE
=====
Prospective holdout = 400 / 400

```



```

Prospective holdout accuracy = 1.
Exhaustive length 0..5 = 1365 / 1365
Frozen hash unchanged = True
=====
POST-HOC TRUE S5 ISOMORPHISM AUDIT
=====
Post-hoc true S5 states = 120
Distinct true S5 behavior signatures = 120
Learned -> true functional = True
True -> learned functional = True
State-space bijection = True
Start-state correspondence = True
Transition correspondence = 480 / 480
Complete state/query behavior = 3000 / 3000
Synchronized product reachable pairs = 120
Product learned->true functional = True
Product true->learned functional = True
Direct behavioral encoding = 120 / 120
=====
S114A COMPLETE
States discovered = 120
Distinct behaviors = 120
Transitions = 480 / 480
Full-signature query baseline = 12025
Actual membership queries = 3708
Queries saved = 8317 (69.1642%)
Mean queries / identification = 7.70894
Information lower bound = 6.90689
Prospective holdout = 400 / 400
Exhaustive audit = 1365 / 1365
Post-hoc state bijection = True
Transition correspondence = 480 / 480
Query behavior correspondence = 3000 / 3000
Synchronized product pairs = 120
Direct behavioral encoding = 120 / 120
Exact machine isomorphism = True
All finite-sequence equivalent = True
Diagnosis = PROSPECTIVE_S5_SUCCESS_FROZEN_QUERY_EFFICIENT_POLICY_RECOVERS_THE\

> _EXACT_120_STATE_S5_MACHINE_WITH_STRONG_QUERY_REDUCTION_AND_DIRECT_BEHAV\

> IORAL_ENCODING
=====

```

In [448...

```

(* ===== *)
(* S115A *)
(* PROSPECTIVE NON-INVERTIBLE TRANSFORMATION-SYSTEM DISCOVERY *)
(* *)
(* PURPOSE *)
(* ----- *)
(* Test whether the frozen active discrete-state discovery *)
(* principle generalizes BEYOND permutation groups. *)
(* *)
(* Hidden states are deterministic transformations: *)
(* *)
(* f : {0,1,2,3} -> {0,1,2,3} *)

```

```

(*) *)
(* but f is NOT required to be bijective. *)
(*) *)
(* Therefore hidden computation may contain: *)
(* - rank reduction *)
(* - many-to-one mappings *)
(* - irreversible transitions *)
(* - information destruction *)
(*) *)
(* FROZEN QUERY POLICY *)
(*) ----- *)
(* Same policy as S113B / S114A: *)
(*) *)
(*  $q^* = \operatorname{argmin}_q \max(N_0(q), N_1(q))$  *)
(*) *)
(* LEARNER KNOWS *)
(*) ----- *)
(* - alphabet *)
(* - four query symbols *)
(* - each hidden state is SOME deterministic map *)
(* from four symbols to four symbols *)
(* - oracle returns final binary Y only *)
(*) *)
(* Thus hypothesis class = all  $4^4 = 256$  deterministic maps. *)
(*) *)
(* LEARNER IS NOT GIVEN *)
(*) ----- *)
(* - which mappings are reachable *)
(* - reachable state count *)
(* - hidden state *)
(* - transition table *)
(* - intermediate state *)
(* - rank of hidden state *)
(* - which generator is irreversible *)
(*) *)
(* TRUE SYSTEM IS USED ONLY AFTER FREEZE. *)
(*) ===== *)

ClearAll["Global`S115A*"];
ClearAll["Global`s115A*"];

s115AResult =
Catch[

(

(*) ===== *)
(*) 0. FAIL SAFE *)
(*) ===== *)

S115AFail[msg_] := (
  Print["STOP: ", msg];
  Throw[$Failed, "S115A_STOP"];
);

```

```

Print["====="];
Print["S115A NON-INVERTIBLE TRANSFORMATION DISCOVERY"];
Print["====="];

(* ===== *)
(* 1. TASK INTERFACE *)
(* ===== *)

alphabetS115A =
  {0, 1, 2};

querySymbolsS115A =
  {0, 1, 2, 3};

allQueryPairsS115A =
  Tuples[
    querySymbolsS115A,
    2
  ];

queryIndexS115A =
  AssociationThread[
    querySymbolsS115A,
    Range[4]
  ];

Print[
  "Primitive alphabet = ",
  alphabetS115A
];

Print[
  "Query symbols = ",
  querySymbolsS115A
];

Print[
  "Full observable queries per state = ",
  Length[allQueryPairsS115A]
];

Print["NO BIJECTION PRIOR."];
Print["STATE BEHAVIOR MAY BE MANY-TO-ONE."];
Print["IRREVERSIBLE TRANSITIONS ARE ALLOWED."];
Print["LEARNER IS NOT GIVEN REACHABLE STATE COUNT."];
Print["LEARNER RECEIVES FINAL BINARY Y ONLY."];

```

```

(* ===== *)
(* 2. PROSPECTIVE HOLDOUT *)
(* *)
(* Inputs are generated BEFORE discovery. *)
(* Environment outputs remain sealed until after freeze. *)
(* ===== *)

holdoutSeedS115A =
  1150401;

holdoutDepthsS115A =
  {
    12,
    24,
    48,
    96
  };

holdoutSequencesPerDepthS115A =
  100;

prospectiveHoldoutSeqsS115A =
  BlockRandom[

    SeedRandom[
      holdoutSeedS115A
    ];

    Flatten[
      Table[

        Table[

          <|

            "Depth" ->
              depth,

            "Sequence" ->
              RandomChoice[
                alphabetS115A,
                depth
              ]

          |>,

          {
            holdoutSequencesPerDepthS115A
          }
        ],
      ],
    ]
  ];

```

```

        {
            depth,
            holdoutDepthsS115A
        }
    ],
    1
]
];

Print[
    "Prospective holdout sequences created = ",
    Length[prospectiveHoldoutSeqsS115A]
];

Print[
    "Holdout depths = ",
    holdoutDepthsS115A
];

Print["HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED."];

(* ===== *)
(* 3. HIDDEN ENVIRONMENT *)
(* *)
(* Internal transformation representation uses {1,2,3,4}. *)
(* *)
(* token 0: reversible swap 1<->2 *)
(* token 1: reversible swap 2<->3 *)
(* token 2: NON-INVERTIBLE map *)
(* *)
(*      1 -> 1 *)
(*      2 -> 2 *)
(*      3 -> 1 *)
(*      4 -> 3 *)
(* *)
(* token 2 has rank 3 and destroys information. *)
(* *)
(* Learner cannot read these functions directly. *)
(* ===== *)

S115AEnvOp[0] :=
    {2, 1, 3, 4};

S115AEnvOp[1] :=
    {1, 3, 2, 4};

S115AEnvOp[2] :=
    {1, 2, 1, 3};

```

```

S115AEnvStep[
  hiddenState_List,
  token_Integer
] :=
  S115AEnvOp[token][[
    hiddenState
  ]];

S115AEnvFinalState[
  seq_List
] :=
  Fold[
    S115AEnvStep,
    {1, 2, 3, 4},
    seq
  ];

S115AEnvRawY[
  seq_List,
  start_,
  target_
] :=
Module[
  {
    hiddenState,
    startIndex,
    targetIndex
  },

  hiddenState =
    S115AEnvFinalState[
      seq
    ];

  startIndex =
    Lookup[
      queryIndexS115A,
      start,
      Missing["UnknownStart"]
    ];

  targetIndex =
    Lookup[
      queryIndexS115A,
      target,
      Missing["UnknownTarget"]
    ];

```

```

If[
  MissingQ[startIndex] ||
  MissingQ[targetIndex],

  Return[
    Missing[
      "UnknownQuery"
    ]
  ]
];

Boole[
  hiddenState[[startIndex]] ===
  targetIndex
];

(* ===== *)
(* 4. BINARY MEMBERSHIP ORACLE *)
(* ===== *)

oracleCacheS115A =
  <||>;

oracleUniqueQueryCountS115A =
  0;

oracleTotalCallCountS115A =
  0;

S115AOracleKey[
  seq_List,
  start_,
  target_
] :=
  ToString[
    InputForm[
      {
        seq,
        start,
        target
      }
    ]
  ];

S115AOracleY[
  seq_List,
  start_,
  target_
] :=

```

```
Module[
  {
    key,
    y
  },

  oracleTotalCallCountS115A++;

  key =
    S115AOracleKey[
      seq,
      start,
      target
    ];

  If[
    KeyExistsQ[
      oracleCacheS115A,
      key
    ],

    Return[
      Lookup[
        oracleCacheS115A,
        key
      ]
    ]
  ];

  y =
    S115AEnvRawY[
      seq,
      start,
      target
    ];

  If[
    !MemberQ[
      {0, 1},
      y
    ],

    S115AFail[
      "Environment returned non-binary output."
    ]
  ];

  oracleUniqueQueryCountS115A++;
```



```

AssociateTo[
  oracleCacheS115A,
  key -> y
];

y
];

(* ===== *)
(* 5. GENERAL DETERMINISTIC-MAPPING HYPOTHESIS CLASS *)
(* *)
(* IMPORTANT DIFFERENCE FROM S113/S114: *)
(* *)
(* NOT Permutations[]. *)
(* *)
(* Every of the four inputs independently maps to one of four *)
(* outputs: *)
(* *)
(* 4^4 = 256 possible deterministic behaviors. *)
(* *)
(* Most need NOT be reachable. *)
(* ===== *)

behaviorHypothesesS115A =
  Tuples[
    querySymbolsS115A,
    Length[querySymbolsS115A]
  ];

behaviorHypothesisCountS115A =
  Length[
    behaviorHypothesesS115A
  ];

Print[
  "General deterministic behavior hypotheses = ",
  behaviorHypothesisCountS115A
];

If[
  behaviorHypothesisCountS115A != 256,

  S115AFail[
    "Expected 4^4 = 256 deterministic behavior hypotheses."
  ]
];

S115ACandidateY[
  candidate_List,
  start_,

```

```

target_
] :=
Module[
  {
    startIndex
  },

  startIndex =
    Lookup[
      queryIndexS115A,
      start,
      Missing["UnknownStart"]
    ];

  If[
    MissingQ[startIndex],

    Return[
      Missing[
        "UnknownStart"
      ]
    ]
  ];

  Boole[
    candidate[[startIndex]] ==
    target
  ]
];

S115ASignatureFromBehavior[
  candidate_List
] :=
Table[

  S115ACandidateY[
    candidate,
    pair[[1]],
    pair[[2]]
  ],

  {
    pair,
    allQueryPairsS115A
  }
];

S115ABehaviorKey[
  candidate_List
] :=
ToString[

```

```

    InputForm[
      candidate
    ]
  ];

S115ABehaviorRank[
  behavior_List
] :=
  Length[
    DeleteDuplicates[
      behavior
    ]
  ];

(* ===== *)
(* 6. FROZEN ADAPTIVE QUERY POLICY *)
(* *)
(* Identical decision principle to S113B/S114A: *)
(* *)
(*   choose q minimizing max(N0,N1) *)
(* *)
(* NO special treatment for irreversible states. *)
(* ===== *)

S115AChooseQuery[
  candidates_List,
  askedQueries_List
] :=
Module[
  {
    available,
    scored,
    informative,
    best
  },

  available =
    Select[
      allQueryPairsS115A,

      !MemberQ[
        askedQueries,
        #
      ] &
    ];

  If[
    Length[available] == 0,

    Return[
      Missing[
        "NoAvailableQuery"
      ]
    ]
  ]
];

```

```

    ]
  ]
];

scored =
  Table[

    Module[
      {
        answers,
        n0,
        n1
      },

      answers =
        Table[

          S115ACandidateY[
            candidate,
            query[[1]],
            query[[2]]
          ],

          {
            candidate,
            candidates
          }
        ];

      n0 =
        Count[
          answers,
          0
        ];

      n1 =
        Count[
          answers,
          1
        ];

      <|

        "Query" ->
          query,

        "N0" ->
          n0,

        "N1" ->
          n1,

```

```

        "WorstRemaining" ->
        Max[
            n0,
            n1
        ],

        "Imbalance" ->
        Abs[
            n0 - n1
        ]

        |>

    ],

    {
        query,
        available
    }
];

informative =
  Select[
    scored,

    And[
      #["N0"] > 0,
      #["N1"] > 0
    ] &
  ];

If[
  Length[informative] == 0,

  Return[
    Missing[
      "NoInformativeQuery"
    ]
  ]
];

best =
  First[
    SortBy[
      informative,

      Function[
        row,

        {
          row["WorstRemaining"],
          row["Imbalance"],

```

```

        ToString[
            InputForm[
                row["Query"]
            ]
        ]
    }
]
];

best["Query"]
];

(* ===== *)
(* 7. IDENTIFY ONE PREFIX'S GENERAL DETERMINISTIC BEHAVIOR *)
(* ===== *)

identificationLogS115A =
    {};

identifiedSequenceKeysS115A =
    <||>;

S115AIdentifyBehavior[
    seq_List
] :=
Module[
    {
        candidates,
        asked,
        answers,
        query,
        y,
        beforeCount,
        afterCount,
        result,
        uniqueBefore,
        uniqueAfter,
        seqKey
    },

    candidates =
        behaviorHypothesesS115A;

    asked =
        {};

    answers =
        {};
```

```
uniqueBefore =  
  oracleUniqueQueryCountS115A;  
  
While[  
  Length[candidates] > 1,  
  
  query =  
    S115AChooseQuery[  
      candidates,  
      asked  
    ];  
  
  If[  
    MissingQ[query],  
  
    S115AFail[  
      StringJoin[  
        "Unable to distinguish deterministic behaviors for sequence ",  
        ToString[  
          InputForm[  
            seq  
          ]  
        ]  
      ]  
    ]  
  ];  
  
  beforeCount =  
    Length[  
      candidates  
    ];  
  
  y =  
    S115AOracleY[  
      seq,  
      query[[1]],  
      query[[2]]  
    ];  
  
  AppendTo[  
    asked,  
    query  
  ];  
  
  AppendTo[  
    answers,  
    y
```

```

];

candidates =
  Select[
    candidates,

    S115ACandidateY[
      #,
      query[[1]],
      query[[2]]
    ] ==
    y &
  ];

afterCount =
  Length[
    candidates
  ];

If[
  afterCount == 0,

  S115AFail[
    StringJoin[
      "Observed behavior is inconsistent with deterministic-map hypothesis clas
      ToString[
        InputForm[
          seq
        ]
      ]
    ]
  ];

If[
  afterCount >= beforeCount,

  S115AFail[
    "Adaptive query failed to reduce candidate set."
  ];

];

result =
  First[
    candidates
  ];

uniqueAfter =

```



```

oracleUniqueQueryCountS115A;

seqKey =
  ToString[
    InputForm[
      seq
    ]
  ];

AssociateTo[
  identifiedSequenceKeysS115A,
  seqKey -> True
];

AppendTo[
  identificationLogS115A,

  <|

    "Sequence" ->
      seq,

    "Depth" ->
      Length[seq],

    "QueriesAsked" ->
      Length[asked],

    "UniqueQueriesAdded" ->
      (
        uniqueAfter -
        uniqueBefore
      ),

    "IdentifiedBehavior" ->
      result,

    "BehaviorRank" ->
      S115ABehaviorRank[
        result
      ]

  |>
];

result
];

(* ===== *)
(* 8. ROOT + DISCOVERY STORAGE *)
(* ===== *)

```

```

behaviorToStateS115A =
  <||>;

stateBehaviorS115A =
  <||>;

stateSignatureS115A =
  <||>;

stateRepresentativeSeqS115A =
  <||>;

transitionByStateS115A =
  <||>;

discoveryRowsS115A =
  {};

S115ATransitionKey[
  state_Integer,
  token_Integer
] :=
  ToString[
    InputForm[
      {
        state,
        token
      }
    ]
  ];

rootBehaviorS115A =
  S115AIdentifyBehavior[
    {}
  ];

rootBehaviorKeyS115A =
  S115ABehaviorKey[
    rootBehaviorS115A
  ];

stateCountS115A =
  1;

startStateS115A =

```

```

1;

AssociateTo[
  behaviorToStateS115A,
  rootBehaviorKeyS115A -> 1
];

AssociateTo[
  stateBehaviorS115A,
  1 -> rootBehaviorS115A
];

AssociateTo[
  stateSignatureS115A,
  1 ->
    S115ASignatureFromBehavior[
      rootBehaviorS115A
    ]
];

AssociateTo[
  stateRepresentativeSeqS115A,
  1 -> {}
];

stateQueueS115A =
  {1};

(* ===== *)
(* 9. ACTIVE TRANSITION-CLOSURE DISCOVERY *)
(* ===== *)
(* Search stops ONLY when no new observable state remains. *)
(* ===== *)
(* No reachable-state count is supplied. *)
(* ===== *)

Print["====="];
Print["ACTIVE NON-INVERTIBLE STATE DISCOVERY"];
Print["====="];

discoveryTimingS115A =
  AbsoluteTiming[

    While[
      Length[
        stateQueueS115A
      ] > 0,

```

```
currentStateS115A =  
  First[  
    stateQueueS115A  
  ];  
  
stateQueueS115A =  
  Rest[  
    stateQueueS115A  
  ];  
  
currentRepresentativeS115A =  
  Lookup[  
    stateRepresentativeSeqS115A,  
    currentStateS115A  
  ];  
  
Do[  
  
  successorSeqS115A =  
    Append[  
      currentRepresentativeS115A,  
      token  
    ];  
  
  successorBehaviorS115A =  
    S115AIdentifyBehavior[  
      successorSeqS115A  
    ];  
  
  successorBehaviorKeyS115A =  
    S115ABehaviorKey[  
      successorBehaviorS115A  
    ];  
  
  newStateQ115A =  
    !KeyExistsQ[  
      behaviorToStateS115A,  
      successorBehaviorKeyS115A  
    ];  
  
  If[  
    newStateQ115A,  
  
    stateCountS115A++;  
  
    successorStateS115A =  
      stateCountS115A;
```

```
AssociateTo[
  behaviorToStateS115A,
  successorBehaviorKeyS115A ->
  successorStateS115A
];

AssociateTo[
  stateBehaviorS115A,
  successorStateS115A ->
  successorBehaviorS115A
];

AssociateTo[
  stateSignatureS115A,
  successorStateS115A ->
  S115ASignatureFromBehavior[
    successorBehaviorS115A
  ]
];

AssociateTo[
  stateRepresentativeSeqS115A,
  successorStateS115A ->
  successorSeqS115A
];

AppendTo[
  stateQueueS115A,
  successorStateS115A
],

successorStateS115A =
  Lookup[
    behaviorToStateS115A,
    successorBehaviorKeyS115A
  ]
];

AssociateTo[
  transitionByStateS115A,

  S115ATransitionKey[
    currentStateS115A,
    token
  ] ->
  successorStateS115A
];
```

```

AppendTo[
  discoveryRowsS115A,

  <|

    "SourceState" ->
      currentStateS115A,

    "Token" ->
      token,

    "DestinationState" ->
      successorStateS115A,

    "CreatedNewState" ->
      newStateQ115A

  |>
];

,
{
  token,
  alphabetS115A
}
];

If[
  stateCountS115A > 4096,

  S115AFail[
    "Safety guard exceeded 4096 states."
  ]
];

];

];

Print[
  "Discovery seconds = ",
  N[
    discoveryTimingS115A[[1]]
  ]
];

Print[
  "Observable states discovered = ",
  stateCountS115A
];

```

```

Print[
  "Transitions discovered = ",
  Length[
    transitionByStateS115A
  ]
];

(* ===== *)
(* 10. QUERY EFFICIENCY *)
(* ===== *)

uniqueIdentifiedSequencesS115A =
  Length[
    identifiedSequenceKeysS115A
  ];

fullSignatureBaselineQueriesS115A =
  uniqueIdentifiedSequencesS115A *
  Length[
    allQueryPairsS115A
  ];

membershipQueriesBeforeFreezeS115A =
  oracleUniqueQueryCountS115A;

querySavingsS115A =
  fullSignatureBaselineQueriesS115A -
  membershipQueriesBeforeFreezeS115A;

querySavingsFractionS115A =
  N[
    querySavingsS115A /
    fullSignatureBaselineQueriesS115A
  ];

queriesPerIdentificationS115A =
  (
    #["QueriesAsked"] &
  ) /@
  identificationLogS115A;

meanQueriesPerIdentificationS115A =
  N[
    Mean[
      queriesPerIdentificationS115A
    ]
  ];

```

```
minQueriesPerIdentificationS115A =  
  Min[  
    queriesPerIdentificationS115A  
  ];  
  
maxQueriesPerIdentificationS115A =  
  Max[  
    queriesPerIdentificationS115A  
  ];  
  
queryCountHistogramS115A =  
  Counts[  
    queriesPerIdentificationS115A  
  ];  
  
informationLowerBoundHypothesisS115A =  
  N[  
    Log[  
      2,  
      behaviorHypothesisCountS115A  
    ]  
  ];  
  
Print[  
  "Unique identified sequences = ",  
  uniqueIdentifiedSequencesS115A  
];  
  
Print[  
  "Full-16-query baseline = ",  
  fullSignatureBaselineQueriesS115A  
];  
  
Print[  
  "Actual membership queries = ",  
  membershipQueriesBeforeFreezeS115A  
];  
  
Print[  
  "Query savings = ",  
  querySavingsS115A,  
  " ("  
  100. querySavingsFractionS115A,  
  "%)"  
];  
  
Print[  
  "Mean queries / identification = ",
```



```

meanQueriesPerIdentificationS115A
];

Print[
  "Min / Max queries = ",
  minQueriesPerIdentificationS115A,
  " / ",
  maxQueriesPerIdentificationS115A
];

Print[
  "Query histogram = ",
  queryCountHistogramS115A
];

Print[
  "Hypothesis-class information bound log2(256) = ",
  informationLowerBoundHypothesisS115A
];

(* ===== *)
(* 11. LEARNED RANK STRUCTURE *)
(* ===== *)
(* This is derived ONLY from discovered observable behavior. *)
(* ===== *)

learnedRankHistogramS115A =
  Counts[
    S115ABehaviorRank /@
    Values[
      stateBehaviorS115A
    ]
  ];

learnedNonBijectiveStateCountS115A =
  Count[
    Values[
      stateBehaviorS115A
    ],
    behavior_ /;
    S115ABehaviorRank[
      behavior
    ] < 4
  ];

Print[
  "Learned behavior-rank histogram = ",
  learnedRankHistogramS115A
];

```

```

Print[
  "Learned non-bijective states = ",
  learnedNonBijectiveStateCountS115A,
  " / ",
  stateCountS115A
];

(* ===== *)
(* 12. TRANSITION COMPLETENESS + INJECTIVITY AUDIT *)
(* ===== *)

expectedTransitionCountS115A =
  stateCountS115A *
  Length[
    alphabetS115A
  ];

transitionCompletenessS115A =
  N[
    Length[
      transitionByStateS115A
    ] /
    expectedTransitionCountS115A
  ];

transitionInjectivityTableS115A =
  Table[

    Module[
      {
        destinations,
        uniqueDestinations
      },

      destinations =
        Table[

          Lookup[
            transitionByStateS115A,

            S115ATransitionKey[
              state,
              token
            ]
          ],

          {
            state,
            Range[
              stateCountS115A
            ]
          }
        ]
    ]
  ];

```

```

    ]
  }
];

uniqueDestinations =
  Length[
    DeleteDuplicates[
      destinations
    ]
  ];

<|

  "Token" ->
    token,

  "SourceStates" ->
    stateCountS115A,

  "UniqueDestinations" ->
    uniqueDestinations,

  "Injective" ->
    (
      uniqueDestinations ==
        stateCountS115A
    ),

  "StateMerges" ->
    (
      stateCountS115A -
        uniqueDestinations
    )

|>

],

{
  token,
  alphabetS115A
}
];

nonInjectivePrimitiveCountS115A =
  Count[
    (
      TrueQ[
        !#["Injective"]
      ] &
    ) /@
      transitionInjectivityTableS115A,
    True

```

```

];

Print[
  "Transitions = ",
  Length[
    transitionByStateS115A
  ],
  " / ",
  expectedTransitionCountS115A
];

Print[
  "Transition completeness = ",
  transitionCompletenessS115A
];

Print[
  "Non-injective primitive transition maps = ",
  nonInjectivePrimitiveCountS115A,
  " / ",
  Length[
    alphabetS115A
  ]
];

If[
  Abs[
    transitionCompletenessS115A - 1.
  ] > 10^-12,
  S115AFail[
    "Discovered transition table is incomplete."
  ]
];

(* ===== *)
(* 13. MACHINE EXECUTION *)
(* ===== *)

S115AExecuteState[
  seq_List
] :=
Module[
  {
    state
  },

  state =
    startStateS115A;

```

```

Do[

  state =
    Lookup[
      transitionByStateS115A,

      S115ATransitionKey[
        state,
        token
      ],

      Missing[
        "UnknownTransition"
      ]
    ];

  If[
    MissingQ[state],

    Return[state]
  ];

,
{
  token,
  seq
}
];

state
];

S115ALearnedSignature[
  seq_List
] :=
Module[
{
  state
},

  state =
    S115AExecuteState[
      seq
    ];

  If[
    MissingQ[state],

    Return[
      Missing[

```

```

        "UnknownTransition"
    ]
]
];

Lookup[
    stateSignatureS115A,
    state,
    Missing[
        "UnknownState"
    ]
]
];

(* ===== *)
(* 14. HOLDOUT LEAKAGE AUDIT *)
(* ===== *)

S115AHoldoutTouchedQ[
    seq_List
] :=
AnyTrue[
    allQueryPairsS115A,

    Function[
        pair,

        KeyExistsQ[
            oracleCacheS115A,

            S115AOracleKey[
                seq,
                pair[[1]],
                pair[[2]]
            ]
        ]
    ]
]
];

holdoutTouchedBeforeFreezeS115A =
Count[
    (
        S115AHoldoutTouchedQ[
            #["Sequence"]
        ] &
    ) /@
    prospectiveHoldoutSeqsS115A,
    True
];

Print[
    "Holdout sequences touched before freeze = ",

```

```

holdoutTouchedBeforeFreezeS115A,
" / ",
Length[
    prospectiveHoldoutSeqsS115A
]
];

(* ===== *)
(* 15. FREEZE *)
(* ===== *)

machineHashS115A =
Hash[
{
    stateBehaviorS115A,
    stateSignatureS115A,
    stateRepresentativeSeqS115A,
    transitionByStateS115A,
    startStateS115A
},

"SHA256",
"HexString"
];

Print["====="];
Print["S115A MACHINE FROZEN"];

Print[
    "States = ",
    stateCountS115A
];

Print[
    "Transitions = ",
    Length[
        transitionByStateS115A
    ]
];

Print[
    "Membership queries = ",
    membershipQueriesBeforeFreezeS115A
];

Print[
    "Full-signature baseline = ",
    fullSignatureBaselineQueriesS115A
];

```

```

Print[
  "Query savings = ",
  100. querySavingsFractionS115A,
  "%"
];

Print[
  "Machine hash = ",
  machineHashS115A
];

Print["NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT."];
Print["====="];

(* ===== *)
(* 16. PROSPECTIVE HOLDOUT AFTER FREEZE *)
(* ===== *)

Print["====="];
Print["OPENING NON-INVERTIBLE PROSPECTIVE HOLDOUT"];
Print["====="];

holdoutRowsS115A =
  Table[
    Module[
      {
        seq,
        learnedSignature,
        oracleSignature
      },
      seq =
        row[
          "Sequence"
        ];

      learnedSignature =
        S115ALearnedSignature[
          seq
        ];

      oracleSignature =
        Table[
          S115AOracleY[
            seq,
            pair[[1]],

```



```

        pair[[2]]
      ],

      {
        pair,
        allQueryPairsS115A
      }
    ];

<|

  "Depth" ->
    row["Depth"],

  "LearnedState" ->
    S115AExecuteState[
      seq
    ],

  "SignatureMatch" ->
    (
      learnedSignature ===
      oracleSignature
    )

|>

],

{
  row,
  prospectiveHoldoutSeqsS115A
}
];

holdoutMatchesS115A =
  Count[
    (
      TrueQ[
        #["SignatureMatch"]
      ] &
    ) /@
      holdoutRowsS115A,
    True
  ];

holdoutAccuracyS115A =
  N[
    holdoutMatchesS115A /
    Length[
      holdoutRowsS115A
    ]
  ];

```

```

holdoutPerDepthS115A =
  Table[

    Module[
      {
        rows,
        matches
      },

      rows =
        Select[
          holdoutRowsS115A,
          #["Depth"] === depth &
        ];

      matches =
        Count[
          (
            TrueQ[
              #["SignatureMatch"]
            ] &
          ) /@
            rows,
          True
        ];

      <|

        "Depth" ->
          depth,

        "Sequences" ->
          Length[
            rows
          ],

        "Matches" ->
          matches,

        "Accuracy" ->
          N[
            matches /
              Length[
                rows
              ]
          ]

      |>

    ],

```

```

    {
      depth,
      holdoutDepthsS115A
    }
  ];

Print[
  "Prospective holdout = ",
  holdoutMatchesS115A,
  " / ",
  Length[
    holdoutRowsS115A
  ]
];

(* ===== *)
(* 17. EXHAUSTIVE SHORT-SEQUENCE AUDIT *)
(* *)
(* Alphabet = 3. *)
(* Lengths 0..6 => 1093 sequences. *)
(* ===== *)

auditMaxLengthS115A =
  6;

auditSequencesS115A =
  Flatten[
    Table[
      Tuples[
        alphabetS115A,
        len
      ],
      {
        len,
        0,
        auditMaxLengthS115A
      }
    ],
    1
  ];

auditRowsS115A =
  Table[
    Module[
      {
        learnedSignature,
        oracleSignature
      },

```

```

learnedSignature =
  S115ALearnedSignature[
    seq
  ];

oracleSignature =
  Table[

    S115AOracleY[
      seq,
      pair[[1]],
      pair[[2]]
    ],

    {
      pair,
      allQueryPairsS115A
    }
  ];

<|

  "Length" ->
    Length[
      seq
    ],

  "LearnedState" ->
    S115AExecuteState[
      seq
    ],

  "SignatureMatch" ->
    (
      learnedSignature ===
      oracleSignature
    )

|>

],

{
  seq,
  auditSequencesS115A
}
];

auditMatchesS115A =
  Count[
    (

```

```

    TrueQ[
      #["SignatureMatch"]
    ] &
  ) /@
    auditRowsS115A,
  True
];

auditAccuracyS115A =
N[
  auditMatchesS115A /
  Length[
    auditRowsS115A
  ]
];

auditPerLengthS115A =
Table[

  Module[
    {
      rows,
      matches
    },

    rows =
      Select[
        auditRowsS115A,
        #["Length"] === len &
      ];

    matches =
      Count[
        (
          TrueQ[
            #["SignatureMatch"]
          ] &
        ) /@
          rows,
        True
      ];

    <|

      "Length" ->
        len,

      "Sequences" ->
        Length[
          rows
        ],
    >|
  ],
];

```

```

        "Matches" ->
            matches,

        "Accuracy" ->
            N[
                matches /
                Length[
                    rows
                ]
            ]

        |>

    ],

    {
        len,
        0,
        auditMaxLengthS115A
    }
];

Print[
    "Exhaustive audit = ",
    auditMatchesS115A,
    " / ",
    Length[
        auditRowsS115A
    ]
];

(* ===== *)
(* 18. FROZEN HASH *)
(* ===== *)

recomputedHashS115A =
    Hash[
        {
            stateBehaviorS115A,
            stateSignatureS115A,
            stateRepresentativeSeqS115A,
            transitionByStateS115A,
            startStateS115A
        },

        "SHA256",
        "HexString"
    ];

frozenHashMatchesS115A =
    recomputedHashS115A ==
    machineHashS115A;

```

```

Print[
  "Frozen hash unchanged = ",
  frozenHashMatchesS115A
];

If[
  !TrueQ[
    frozenHashMatchesS115A
  ],

  S115AFail[
    "Machine changed after freeze."
  ]
];

(* ===== *)
(* 19. POST-HOC TRUE REACHABLE-STATE CLOSURE *)
(* *)
(* IMPORTANT: only NOW is the hidden state graph enumerated. *)
(* *)
(* Discovery did not know its size. *)
(* ===== *)

Print["====="];
Print["POST-HOC TRUE NON-INVERTIBLE SYSTEM AUDIT"];
Print["====="];

S115ATrueStateKey[
  hiddenState_List
] :=
  ToString[
    InputForm[
      hiddenState
    ]
  ];

S115AEnumerateTrueClosure[] :=
Module[
  {
    seen,
    queue,
    current,
    next,
    key
  },

  seen =
    <|
      S115ATrueStateKey[

```

```

        {1, 2, 3, 4}
    ] ->
    {1, 2, 3, 4}
|>;

queue =
{
    {1, 2, 3, 4}
};

While[
    Length[
        queue
    ] > 0,

    current =
        First[
            queue
        ];

    queue =
        Rest[
            queue
        ];

    Do[

        next =
            S115AEnvStep[
                current,
                token
            ];

        key =
            S115ATrueStateKey[
                next
            ];

        If[
            !KeyExistsQ[
                seen,
                key
            ],

            AssociateTo[
                seen,
                key -> next
            ];

```



```

        AppendTo[
            queue,
            next
        ];
    ],
    {
        token,
        alphabetS115A
    }
];

];

Values[
    seen
]
];

trueReachableStatesS115A =
    S115AEnumerateTrueClosure[];

trueStateCountS115A =
    Length[
        trueReachableStatesS115A
    ];

trueStartStateS115A =
    {1, 2, 3, 4};

Print[
    "Post-hoc true reachable states = ",
    trueStateCountS115A
];

(* ===== *)
(* 20. TRUE RANK HISTOGRAM *)
(* ===== *)

S115ATrueRank[
    hiddenState_List
] :=
    Length[
        DeleteDuplicates[
            hiddenState
        ]
    ];

```

```

trueRankHistogramS115A =
  Counts[
    S115ATrueRank /@
      trueReachableStatesS115A
  ];

trueNonBijectiveStateCountS115A =
  Count[
    trueReachableStatesS115A,

    hiddenState_ /;
      S115ATrueRank[
        hiddenState
      ] < 4
  ];

Print[
  "True rank histogram = ",
  trueRankHistogramS115A
];

Print[
  "True non-bijective states = ",
  trueNonBijectiveStateCountS115A,
  " / ",
  trueStateCountS115A
];

rankHistogramMatchesS115A =
  learnedRankHistogramS115A ==
    trueRankHistogramS115A;

Print[
  "Learned/true rank histogram match = ",
  rankHistogramMatchesS115A
];

(* ===== *)
(* 21. TRUE QUERY SIGNATURE *)
(* ===== *)

S115ATrueQuery[
  hiddenState_List,
  start_,
  target_
] :=
Module[
  {
    startIndex,
    targetIndex
  }

```

```

    },

    startIndex =
      Lookup[
        queryIndexS115A,
        start
      ];

    targetIndex =
      Lookup[
        queryIndexS115A,
        target
      ];

    Boole[
      hiddenState[[startIndex]] ==
        targetIndex
    ]
  ];

  S115ATrueSignature[
    hiddenState_List
  ] :=
    Table[

      S115ATrueQuery[
        hiddenState,
        pair[[1]],
        pair[[2]]
      ],

      {
        pair,
        allQueryPairsS115A
      }
    ];

  trueDistinctSignatureCountS115A =
    Length[
      DeleteDuplicates[
        S115ATrueSignature /@
          trueReachableStatesS115A
      ]
    ];

  Print[
    "Distinct true observable signatures = ",
    trueDistinctSignatureCountS115A
  ];

```

```

(* ===== *)
(* 22. LEARNED -> TRUE STATE CORRESPONDENCE *)
(* ===== *)

stateCorrespondenceS115A =
  Table[

    Module[
      {
        learnedSignature,
        matches
      },

      learnedSignature =
        Lookup[
          stateSignatureS115A,
          learnedState
        ];

      matches =
        Select[
          trueReachableStatesS115A,

          S115ATrueSignature[#] ===
            learnedSignature &
        ];

      <|

        "LearnedState" ->
          learnedState,

        "RepresentativeSeq" ->
          Lookup[
            stateRepresentativeSeqS115A,
            learnedState
          ],

        "LearnedBehavior" ->
          Lookup[
            stateBehaviorS115A,
            learnedState
          ],

        "LearnedRank" ->
          S115ABehaviorRank[
            Lookup[
              stateBehaviorS115A,
              learnedState
            ]
          ],

      ]
    ]
  ];

```

```

      "TrueMatches" ->
        matches,

      "MatchCount" ->
        Length[
          matches
        ],

      "Functional" ->
        (
          Length[matches] == 1
        )

    |>

  ],

  {
    learnedState,
    Range[
      stateCountS115A
    ]
  }
];

learnedToTrueFunctionalS115A =
  AllTrue[
    stateCorrespondenceS115A,

    Function[
      row,
      TrueQ[
        row["Functional"]
      ]
    ]
  ];

assignedTrueStatesS115A =
  If[
    learnedToTrueFunctionalS115A,

    (
      First[
        #["TrueMatches"]
      ] &
    ) /@
      stateCorrespondenceS115A,

    {}
  ];

trueToLearnedFunctionalS115A =
  If[

```

```

learnedToTrueFunctionalS115A,

AllTrue[
  trueReachableStatesS115A,

  Function[
    trueState,

    Count[
      assignedTrueStatesS115A,
      trueState
    ] == 1
  ]
],

False
];

stateBijectionPassedS115A =
And[
  learnedToTrueFunctionalS115A,
  trueToLearnedFunctionalS115A,
  stateCountS115A ==
    trueStateCountS115A
];

Print[
  "Learned -> true functional = ",
  learnedToTrueFunctionalS115A
];

Print[
  "True -> learned functional = ",
  trueToLearnedFunctionalS115A
];

Print[
  "State-space bijection = ",
  stateBijectionPassedS115A
];

(* ===== *)
(* 23. BUILD POST-HOC PHI *)
(* ===== *)

phiS115A =
If[
  stateBijectionPassedS115A,

  Association[
    Table[

```

```

    row["LearnedState"] ->
      First[
        row["TrueMatches"]
      ],
    {
      row,
      stateCorrespondenceS115A
    }
  ]
],
<||>
];

startStateMatchS115A =
If[
  stateBijectionPassedS115A,

  Lookup[
    phiS115A,
    startStateS115A,
    Missing[
      "NoMapping"
    ]
  ] ==
    trueStartStateS115A,

  False
];

Print[
  "Start-state correspondence = ",
  startStateMatchS115A
];

(* ===== *)
(* 24. EXACT TRANSITION ISOMORPHISM *)
(* ===== *)

S115ALearnedStep[
  learnedState_Integer,
  token_Integer
] :=
Lookup[
  transitionByStateS115A,

  S115ATransitionKey[
    learnedState,
    token
  ],

```

```
Missing[
  "UnknownTransition"
]
];

transitionAuditRowsS115A =
  Flatten[
    Table[

      Module[
        {
          learnedDestination,
          trueSource,
          mappedLearnedDestination,
          trueDestination
        },

        learnedDestination =
          S115ALearnedStep[
            learnedState,
            token
          ];

        trueSource =
          Lookup[
            phiS115A,
            learnedState,
            Missing[
              "NoPhi"
            ]
          ];

        mappedLearnedDestination =
          If[
            IntegerQ[
              learnedDestination
            ],

            Lookup[
              phiS115A,
              learnedDestination,
              Missing[
                "NoDestinationPhi"
              ]
            ],

            Missing[
              "NoLearnedDestination"
            ]
          ];
      ],
    ];
```



```

trueDestination =
  If[
    ListQ[
      trueSource
    ],

    S115AEnvStep[
      trueSource,
      token
    ],

    Missing[
      "NoTrueSource"
    ]
  ];

<|

  "LearnedSource" ->
    learnedState,

  "Token" ->
    token,

  "LearnedDestination" ->
    learnedDestination,

  "TrueSource" ->
    trueSource,

  "MappedLearnedDestination" ->
    mappedLearnedDestination,

  "TrueDestination" ->
    trueDestination,

  "Match" ->
    (
      mappedLearnedDestination ===
      trueDestination
    )

|>

],

{
  learnedState,
  Range[
    stateCountS115A
  ]
},

{
  token,

```

```

        alphabetS115A
    }
],

1
];

transitionTotalS115A =
    Length[
        transitionAuditRowsS115A
    ];

transitionMatchesS115A =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        transitionAuditRowsS115A,
        True
    ];

transitionIsomorphismPassedS115A =
    transitionMatchesS115A ==
        transitionTotalS115A;

Print[
    "Transition correspondence = ",
    transitionMatchesS115A,
    " / ",
    transitionTotalS115A
];

(* ===== *)
(* 25. COMPLETE STATE x QUERY AUDIT *)
(* ===== *)

queryAuditRowsS115A =
    Flatten[
        Table[

            Module[
                {
                    trueState,
                    learnedBehavior
                },

                trueState =
                    Lookup[

```

```

    phiS115A,
    learnedState,
    Missing[
        "NoPhi"
    ]
];

learnedBehavior =
    Lookup[
        stateBehaviorS115A,
        learnedState
    ];

Table[

    Module[
        {
            learnedY,
            trueY
        },

        learnedY =
            S115ACandidateY[
                learnedBehavior,
                pair[[1]],
                pair[[2]]
            ];

        trueY =
            If[
                ListQ[
                    trueState
                ],

                S115ATrueQuery[
                    trueState,
                    pair[[1]],
                    pair[[2]]
                ],

                Missing[
                    "NoTrueState"
                ]
            ];

        <|

            "LearnedState" ->
                learnedState,

            "Start" ->

```

```

        pair[[1]],

        "Target" ->
        pair[[2]],

        "LearnedY" ->
        learnedY,

        "TrueY" ->
        trueY,

        "Match" ->
        (
            learnedY ==
            trueY
        )

    |>

],

{
    pair,
    allQueryPairsS115A
}

],

{
    learnedState,
    Range[
        stateCountS115A
    ]
}

],

1
];

queryTotalS115A =
    Length[
        queryAuditRowsS115A
    ];

queryMatchesS115A =
    Count[
        (
            TrueQ[
                #["Match"]
            ] &
        ) /@
        queryAuditRowsS115A,
        True
    ]

```

```

];

fullQueryBehaviorPassedS115A =
  queryMatchesS115A ==
    queryTotalS115A;

Print[
  "Complete state/query behavior = ",
  queryMatchesS115A,
  " / ",
  queryTotalS115A
];

(* ===== *)
(* 26. SYNCHRONIZED PRODUCT CLOSURE *)
(* ===== *)

S115AProductClosure[] :=
Module[
  {
    startPair,
    reached,
    frontier,
    nextPairs,
    newPairs,
    learnedNext,
    trueNext,
    guard
  },

  startPair =
  {
    startStateS115A,
    trueStartStateS115A
  };

  reached =
  {
    startPair
  };

  frontier =
  {
    startPair
  };

  guard =
  0;

```

```

While[
  Length[
    frontier
  ] > 0,

  guard++;

  If[
    guard > 5000,

    Return[$Failed]
  ];

  nextPairs =
    DeleteDuplicates[
      Flatten[
        Table[

          learnedNext =
            S115ALearnedStep[
              pair[[1]],
              token
            ];

          trueNext =
            S115AEnvStep[
              pair[[2]],
              token
            ];

          {
            learnedNext,
            trueNext
          },

          {
            pair,
            frontier
          },

          {
            token,
            alphabetS115A
          }
        ],
        1
      ]
    ];

```

```

If[
  AnyTrue[
    nextPairs,

    Function[
      pair,
      !IntegerQ[
        pair[[1]]
      ]
    ]
  ],

  Return[$Failed]
];

newPairs =
  Select[
    nextPairs,

    Function[
      pair,
      !MemberQ[
        reached,
        pair
      ]
    ]
  ];

reached =
  Join[
    reached,
    newPairs
  ];

frontier =
  newPairs;

];

reached
];

productPairsS115A =
  S115AProductClosure[];

If[
  productPairsS115A === $Failed,

  S115AFail[

```

```

    "Synchronized product exploration failed."
  ]
];

productPairCountS115A =
  Length[
    productPairsS115A
  ];

productLearnedFunctionalS115A =
  AllTrue[
    Range[
      stateCountS115A
    ],
    Function[
      learnedState,

      Length[
        DeleteDuplicates[
          (
            #[[2]] &
          ) /@
          Select[
            productPairsS115A,
            #[[1]] ===
              learnedState &
          ]
        ]
      ] == 1
    ]
  ];

productTrueFunctionalS115A =
  AllTrue[
    trueReachableStatesS115A,

    Function[
      trueState,

      Length[
        DeleteDuplicates[
          (
            #[[1]] &
          ) /@
          Select[
            productPairsS115A,
            #[[2]] ===
              trueState &
          ]
        ]
      ] == 1
    ]
  ]

```



```

];

productBijectionPassedS115A =
  And[
    productPairCountS115A ==
      stateCountS115A,

    productLearnedFunctionalS115A,

    productTrueFunctionalS115A
  ];

Print[
  "Synchronized product pairs = ",
  productPairCountS115A
];

Print[
  "Product learned->true functional = ",
  productLearnedFunctionalS115A
];

Print[
  "Product true->learned functional = ",
  productTrueFunctionalS115A
];

(* ===== *)
(* 27. DIRECT BEHAVIORAL ENCODING *)
(* *)
(* Learner behavior uses {0,1,2,3}. *)
(* True transformation uses {1,2,3,4}. *)
(* ===== *)

directEncodingRowsS115A =
  Table[

    Module[
      {
        learnedBehavior,
        mappedTrue
      },

      learnedBehavior =
        Lookup[
          stateBehaviorS115A,
          learnedState
        ];
    ],
  ];

```

```

mappedTrue =
  Lookup[
    phiS115A,
    learnedState,
    Missing[
      "NoPhi"
    ]
  ];

<|

"LearnedState" ->
  learnedState,

"LearnedBehavior0123" ->
  learnedBehavior,

"Converted1234" ->
  (
    learnedBehavior + 1
  ),

"MappedTrueState" ->
  mappedTrue,

"Rank" ->
  S115ABehaviorRank[
    learnedBehavior
  ],

"Match" ->
  (
    learnedBehavior + 1 ==
    mappedTrue
  )

|>

],

{
  learnedState,
  Range[
    stateCountS115A
  ]
}
];

directEncodingMatchesS115A =
  Count[
    (
      TrueQ[
        #["Match"]
      ] &
    )
  ]

```

```

    ) /@
    directEncodingRowsS115A,
    True
];

directEncodingExactS115A =
  directEncodingMatchesS115A ==
    stateCountS115A;

Print[
  "Direct behavioral encoding = ",
  directEncodingMatchesS115A,
  " / ",
  stateCountS115A
];

(* ===== *)
(* 28. GLOBAL EXACTNESS *)
(* ===== *)

exactMachineIsomorphismS115A =
  And[
    frozenHashMatchesS115A,
    stateBijectionPassedS115A,
    startStateMatchS115A,
    transitionIsomorphismPassedS115A,
    fullQueryBehaviorPassedS115A,
    productBijectionPassedS115A
  ];

allFiniteSequenceEquivalentS115A =
  exactMachineIsomorphismS115A;

stateCountCorrectPostHocS115A =
  stateCountS115A ==
    trueStateCountS115A;

irreversibilityConfirmedS115A =
  And[
    trueNonBijectiveStateCountS115A > 0,
    learnedNonBijectiveStateCountS115A > 0,
    nonInjectivePrimitiveCountS115A > 0,
    MemberQ[
      Keys[
        learnedRankHistogramS115A
      ],
      1
    ],
    MemberQ[
      Keys[

```

```

        learnedRankHistogramS115A
    ],
    2
],
MemberQ[
    Keys[
        learnedRankHistogramS115A
    ],
    3
]
];

prospectiveStrongS115A =
And[
    holdoutTouchedBeforeFreezeS115A == 0,
    stateCountCorrectPostHocS115A,
    rankHistogramMatchesS115A,
    irreversibilityConfirmedS115A,
    Abs[
        transitionCompletenessS115A - 1.
    ] < 10^-12,
    Abs[
        holdoutAccuracyS115A - 1.
    ] < 10^-12,
    Abs[
        auditAccuracyS115A - 1.
    ] < 10^-12,
    exactMachineIsomorphismS115A
];

diagnosisS115A =
Which[

    prospectiveStrongS115A &&
    directEncodingExactS115A,

        "PROSPECTIVE_NONINVERTIBLE_SUCCESS_ACTIVE_FINAL_OUTPUT_LEARNING_RECOVERS_THE_

!irreversibilityConfirmedS115A,

        "BENCHMARK_DID_NOT_ESTABLISH_THE_REQUIRED_NONINVERTIBLE_RANK_REDUCING_STRUCTU

!stateCountCorrectPostHocS115A,

        "NONINVERTIBLE_DISCOVERY_DOES_NOT_RECOVER_THE_COMPLETE_REACHABLE_STATE_SPACE"

!exactMachineIsomorphismS115A,

        "DISCOVERED_NONINVERTIBLE_MACHINE_IS_NOT_EXACTLY_ISOMORPHIC_TO_THE_TRUE_SYSTE

```

```

holdoutAccuracyS115A < 1.,

    "NONINVERTIBLE_MACHINE_FAILS_PROSPECTIVE_LONG_SEQUENCE_HOLDOUT",

    True,

    "INCONCLUSIVE"
];

(* ===== *)
(* 29. FINAL REPORT *)
(* ===== *)

finalS115A =
<|
    "Stage" ->
        "S115A_ProspectiveNonInvertibleTransformationDiscovery",

    "BijectionPriorUsed" ->
        False,

    "GeneralDeterministicMapPriorUsed" ->
        True,

    "BehaviorHypothesisCount" ->
        behaviorHypothesisCountS115A,

    "HiddenReachableStateCountUsedForStopping" ->
        False,

    "LearnerReceivesIntermediateTrueState" ->
        False,

    "LearnerReceivesFinalBinaryYOnly" ->
        True,

    "StatesDiscovered" ->
        stateCountS115A,

    "TransitionsDiscovered" ->
        Length[
            transitionByStateS115A
        ],

    "TransitionCompleteness" ->
        transitionCompletenessS115A,

    "LearnedRankHistogram" ->
        learnedRankHistogramS115A,

    "LearnedNonBijectiveStates" ->
        learnedNonBijectiveStateCountS115A,

```

```
"NonInjectivePrimitiveTransitions" ->
  nonInjectivePrimitiveCountS115A,

"UniqueIdentifiedSequences" ->
  uniqueIdentifiedSequencesS115A,

"FullSignatureBaselineQueries" ->
  fullSignatureBaselineQueriesS115A,

"ActualMembershipQueries" ->
  membershipQueriesBeforeFreezeS115A,

"QueriesSaved" ->
  querySavingsS115A,

"QuerySavingsFraction" ->
  querySavingsFractionS115A,

"MeanQueriesPerIdentification" ->
  meanQueriesPerIdentificationS115A,

"MinQueriesPerIdentification" ->
  minQueriesPerIdentificationS115A,

"MaxQueriesPerIdentification" ->
  maxQueriesPerIdentificationS115A,

"QueryHistogram" ->
  queryCountHistogramS115A,

"HoldoutTouchedBeforeFreeze" ->
  holdoutTouchedBeforeFreezeS115A,

"HoldoutMatches" ->
  holdoutMatchesS115A,

"HoldoutAccuracy" ->
  holdoutAccuracyS115A,

"ExhaustiveSequences" ->
  Length[
    auditRowsS115A
  ],

"ExhaustiveMatches" ->
  auditMatchesS115A,

"ExhaustiveAccuracy" ->
  auditAccuracyS115A,

"TrueReachableStatesPostHoc" ->
  trueStateCountS115A,

"TrueRankHistogramPostHoc" ->
  trueRankHistogramS115A,
```

```
"RankHistogramMatches" ->
  rankHistogramMatchesS115A,

"IrreversibilityConfirmed" ->
  irreversibilityConfirmedS115A,

"StateBijectionPassed" ->
  stateBijectionPassedS115A,

"StartStateMatch" ->
  startStateMatchS115A,

"TransitionMatches" ->
  transitionMatchesS115A,

"TransitionTotal" ->
  transitionTotalS115A,

"TransitionIsomorphismPassed" ->
  transitionIsomorphismPassedS115A,

"QueryBehaviorMatches" ->
  queryMatchesS115A,

"QueryBehaviorTotal" ->
  queryTotalS115A,

"ProductPairs" ->
  productPairCountS115A,

"ProductBijectionPassed" ->
  productBijectionPassedS115A,

"DirectEncodingMatches" ->
  directEncodingMatchesS115A,

"DirectEncodingExact" ->
  directEncodingExactS115A,

"ExactMachineIsomorphism" ->
  exactMachineIsomorphismS115A,

"AllFiniteSequenceEquivalent" ->
  allFiniteSequenceEquivalentS115A,

"ProspectiveStrong" ->
  prospectiveStrongS115A,

"Diagnosis" ->
  diagnosisS115A,

"MachineHash" ->
  machineHashS115A
```

```
|>;
```

```
Print["====="];
Print["S115A COMPLETE"];

Print[
  "States discovered = ",
  stateCountS115A
];

Print[
  "Transitions = ",
  Length[
    transitionByStateS115A
  ],
  " / ",
  expectedTransitionCountS115A
];

Print[
  "Learned rank histogram = ",
  learnedRankHistogramS115A
];

Print[
  "Non-bijective learned states = ",
  learnedNonBijectiveStateCountS115A,
  " / ",
  stateCountS115A
];

Print[
  "Non-injective primitive transitions = ",
  nonInjectivePrimitiveCountS115A
];

Print[
  "Full query baseline = ",
  fullSignatureBaselineQueriesS115A
];

Print[
  "Actual membership queries = ",
  membershipQueriesBeforeFreezeS115A
];

Print[
  "Query savings = ",
  100. querySavingsFractionS115A,
```



```
    "%"  
];  
  
Print[  
    "Prospective holdout = ",  
    holdoutMatchesS115A,  
    " / ",  
    Length[  
        holdoutRowsS115A  
    ]  
];  
  
Print[  
    "Exhaustive audit = ",  
    auditMatchesS115A,  
    " / ",  
    Length[  
        auditRowsS115A  
    ]  
];  
  
Print[  
    "Post-hoc true reachable states = ",  
    trueStateCountS115A  
];  
  
Print[  
    "True rank histogram = ",  
    trueRankHistogramS115A  
];  
  
Print[  
    "Rank histogram match = ",  
    rankHistogramMatchesS115A  
];  
  
Print[  
    "State bijection = ",  
    stateBijectionPassedS115A  
];  
  
Print[  
    "Transition correspondence = ",  
    transitionMatchesS115A,  
    " / ",  
    transitionTotalS115A  
];
```

```
Print[
  "Query behavior = ",
  queryMatchesS115A,
  " / ",
  queryTotalS115A
];

Print[
  "Synchronized product pairs = ",
  productPairCountS115A
];

Print[
  "Direct encoding = ",
  directEncodingMatchesS115A,
  " / ",
  stateCountS115A
];

Print[
  "Exact machine isomorphism = ",
  exactMachineIsomorphismS115A
];

Print[
  "All finite-sequence equivalent = ",
  allFiniteSequenceEquivalentS115A
];

Print[
  "Irreversibility confirmed = ",
  irreversibilityConfirmedS115A
];

Print[
  "Diagnosis = ",
  diagnosisS115A
];

Print["====="];

finalS115A

),

"S115A_STOP"
];
```

```

=====
S115A NON-INVERTIBLE TRANSFORMATION DISCOVERY
=====
Primitive alphabet = {0, 1, 2}
Query symbols = {0, 1, 2, 3}
Full observable queries per state = 16
NO BIJECTION PRIOR.
STATE BEHAVIOR MAY BE MANY-TO-ONE.
IRREVERSIBLE TRANSITIONS ARE ALLOWED.
LEARNER IS NOT GIVEN REACHABLE STATE COUNT.
LEARNER RECEIVES FINAL BINARY Y ONLY.
Prospective holdout sequences created = 400
Holdout depths = {12, 24, 48, 96}
HOLDOUT OUTPUTS HAVE NOT BEEN ACCESSED.
General deterministic behavior hypotheses = 256
=====
ACTIVE NON-INVERTIBLE STATE DISCOVERY
=====
Discovery seconds = 18.3167
Observable states discovered = 69
Transitions discovered = 207
Unique identified sequences = 208
Full-16-query baseline = 3328
Actual membership queries = 1503
Query savings = 1825 (54.8377%)
Mean queries / identification = 7.22596
Min / Max queries = 4 / 12
Query histogram = <|9 -> 33, 7 -> 34, 8 -> 27, 5 -> 28, 6 -> 38, 10 -> 20,

> 4 -> 18, 11 -> 8, 12 -> 2|>
Hypothesis-class information bound  $\log_2(256) = 8$ .
Learned behavior-rank histogram = <|4 -> 6, 3 -> 18, 2 -> 42, 1 -> 3|>
Learned non-bijective states = 63 / 69
Transitions = 207 / 207
Transition completeness = 1.
Non-injective primitive transition maps = 1 / 3
Holdout sequences touched before freeze = 0 / 400
=====
S115A MACHINE FROZEN
States = 69
Transitions = 207
Membership queries = 1503
Full-signature baseline = 3328
Query savings = 54.8377%
Machine hash = 12505163e1242d3bef393999456c1ee2bd5925f751b9f0b7a94b0e12f25317\

> ea
NO LEARNED STATE OR TRANSITION CHANGES AFTER THIS POINT.
=====
=====
OPENING NON-INVERTIBLE PROSPECTIVE HOLDOUT
=====
Prospective holdout = 400 / 400
Exhaustive audit = 1093 / 1093
Frozen hash unchanged = True
=====

```

POST-HOC TRUE NON-INVERTIBLE SYSTEM AUDIT

=====

Post-hoc true reachable states = 69
 True rank histogram = <|4 -> 6, 3 -> 18, 2 -> 42, 1 -> 3|>
 True non-bijective states = 63 / 69
 Learned/true rank histogram match = True
 Distinct true observable signatures = 69
 Learned -> true functional = True
 True -> learned functional = True
 State-space bijection = True
 Start-state correspondence = True
 Transition correspondence = 207 / 207
 Complete state/query behavior = 1104 / 1104
 Synchronized product pairs = 69
 Product learned->true functional = True
 Product true->learned functional = True
 Direct behavioral encoding = 69 / 69

=====

S115A COMPLETE

States discovered = 69
 Transitions = 207 / 207
 Learned rank histogram = <|4 -> 6, 3 -> 18, 2 -> 42, 1 -> 3|>
 Non-bijective learned states = 63 / 69
 Non-injective primitive transitions = 1
 Full query baseline = 3328
 Actual membership queries = 1503
 Query savings = 54.8377%
 Prospective holdout = 400 / 400
 Exhaustive audit = 1093 / 1093
 Post-hoc true reachable states = 69
 True rank histogram = <|4 -> 6, 3 -> 18, 2 -> 42, 1 -> 3|>
 Rank histogram match = True
 State bijection = True
 Transition correspondence = 207 / 207
 Query behavior = 1104 / 1104
 Synchronized product pairs = 69
 Direct encoding = 69 / 69
 Exact machine isomorphism = True
 All finite-sequence equivalent = True
 Irreversibility confirmed = True
 Diagnosis = PROSPECTIVE_NONINVERTIBLE_SUCCESS_ACTIVE_FINAL_OUTPUT_LEARNING_RE\

> COVERS_THE_EXACT_IRREVERSIBLE_TRANSFORMATION_SYSTEM_WITH_RANK_REDUCTION_\

> MANY_TO_ONE_DYNAMICS_AND_DIRECT_BEHAVIORAL_ENCODING

=====

In []: